Fondamenti dell'Informatica

Appunti di Michele Castenedoli

13 ottobre 2026

Indice

0 Prerequisiti Matematici **1**
- 0.1 Basi di Logica . . . 1
- 0.2 Insiemistica . . . 2

1 Introduzione **8**
- 1.1 Il problema dell'informatica . . . 8
- 1.2 Quante funzioni esistono? . . . 8
- 1.3 Quanti algoritmi esistono? . . . 10
- 1.4 Il teorema di Cantor . . . 10

2 Induzione **12**
- 2.1 Relazioni ben fondate . . . 12
- 2.2 Principio di induzione . . . 12

3 Linguaggi Formali **14**
- 3.1 Alfabeti e stringhe . . . 14
- 3.2 Linguaggi . . . 15
- 3.3 Funzioni come linguaggi . . . 15
- 3.4 Automi a stati finiti deterministici . . . 16
- 3.5 Automi a stati finiti non deterministici . . . 21
- 3.6 Automi a stati finiti non deterministici con ε-transizioni . . . 26
- 3.7 Operazioni Tra Linguaggi . . . 28
- 3.8 Espressioni Regolari . . . 29
- 3.9 Proprietà dei Linguaggi Regolari . . . 32
- 3.10 Problemi di Decidibilità . . . 33
- 3.11 Relazioni di Equivalenza e Linguaggi . . . 33

4 Esercizi **34**
- 4.1 Automi a stati finiti deterministici . . . 34

0 Prerequisiti Matematici

0.1 Basi di Logica

0.1.1 Linguaggio del Primo Ordine

Un *linguaggio del primo ordine* è formato da quattro tipi di simboli:

1. Simboli di relazione, indicati con le metavariabili p, q, r;
2. Simboli di funzione, indicati con le metavariabili f, g, h;
3. Simboli di costante, indicati con le metavariabili c, d;
4. Simboli logici:
   - le parentesi (e) e la virgola;
   - un insieme numerabile di *variabili*, indicate con le metavariabili x, y, v;
   - i *connettivi logici* e i *quantificatori* $(\wedge, \vee, \neg, \implies \iff, \forall, \exists)$;
   - il simbolo $\bot$, che indica il falso.

I simboli di relazione e di funzione hanno ciascuno la propria *arità*, ovvero il numero di argomenti.

A partire da questi simboli si costruiscono, in ordine, i termini, le formule atomiche e le formule.

Definizione (Termine). *I* termini *sono definiti per induzione:*

1. *ogni variabile è un termine;*
2. *ogni costante è un termine;*
3. *se f è un simbolo di funzione n-ario e $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, t_n)$ è un termine.*

Definizione (Formula atomica). *Le* formule atomiche *sono:*

1. *$p(t_1, \dots, t_n)$, se p è un simbolo di relazione n-ario e $t_1, \dots, t_n$ sono termini;*
2. *il simbolo $\bot$.*

Definizione (Formula). *Le* formule *sono definite per induzione:*

1. *ogni formula atomica è una formula;*
2. *se ϕ è una formula, allora $\neg\phi$ è una formula;*
3. *se ϕ e ψ sono formule, allora $\phi \circ \psi$ è una formula, con $\circ \in \{\wedge, \vee, \implies, \iff\}$;*
4. *se ϕ è una formula e x è una variabile, allora $\forall x\,(\phi)$ e $\exists x\,(\phi)$ sono formule.*

0.1.2 Equivalenze

Si scrive $\phi \equiv \psi$ quando le formule ϕ e ψ sono *equivalenti*, ovvero assumono sempre lo stesso valore di verità.

Due negazioni consecutive si annullano (*doppia negazione*):

$$\neg(\neg\phi) \equiv \phi.$$

Le *leggi di De Morgan* descrivono la negazione di una congiunzione e di una disgiunzione:

$$\neg(\phi \wedge \psi) \equiv (\neg\phi) \vee (\neg\psi), \qquad \neg(\phi \vee \psi) \equiv (\neg\phi) \wedge (\neg\psi).$$

Un'*implicazione* può essere riscritta come disgiunzione, oppure come la sua *contronominale* (l'ultima forma):

$$(\phi \implies \psi) \equiv (\neg\phi) \vee \psi \equiv \big((\neg\psi) \implies (\neg\phi)\big).$$

Infine, la *negazione di un quantificatore* si ottiene scambiandolo con l'altro e negando la formula:

$$\neg\big(\exists x\,(\phi)\big) \equiv \forall x\,(\neg\phi), \qquad \neg\big(\forall x\,(\phi)\big) \equiv \exists x\,(\neg\phi).$$

0.1.3 Abbreviazioni

Per i quantificatori ristretti agli elementi di un insieme S si adottano le seguenti abbreviazioni:

$$(\exists x \in S.\ \phi) \equiv \exists x\,(x \in S \wedge \phi),$$
$$(\forall x \in S.\ \phi) \equiv \forall x\,(x \in S \implies \phi),$$

dove il punto si legge *tale che*.

0.2 Insiemistica

0.2.1 Definizione e Rappresentazione

Un *insieme* è una collezione di elementi senza ripetizioni. Esso può essere rappresentato in due modi. La rappresentazione *intensionale* descrive gli elementi attraverso una loro caratteristica: ad esempio

$$\{x \mid p(x)\}$$

è l'insieme di tutti gli elementi x che rendono vero il predicato p. La rappresentazione *estensionale*, invece, elenca tutti gli elementi dell'insieme, come in $\{1, 2, 3\}$.

Un'altra definizione fondamentale per il resto del corso è quella di sottoinsieme.

Definizione (Sottoinsieme). *$A \subseteq B$, ovvero A è un* sottoinsieme *di B, se ogni elemento di A appartiene anche a B. A è invece un* sottoinsieme proprio *di B, $A \subset B$, se in più B contiene almeno un elemento che non appartiene ad A:*

$$A \subseteq B \iff \forall x\,(x \in A \implies x \in B),$$
$$A \subset B \iff A \subseteq B \wedge \exists x \in B.\ x \notin A.$$

0.2.2 Operazioni tra Insiemi

Le operazioni classiche tra insiemi sono:

- Unione: $A \cup B = \{x \mid x \in A \vee x \in B\}$;
- Intersezione: $A \cap B = \{x \mid x \in A \wedge x \in B\}$;
- Differenza: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$;
- Complemento: $\overline{A} = \{x \in U \mid x \notin A\}$, dove U è l'*insieme universo*, che contiene tutti gli elementi considerati.

Per ogni $x \in U$ vale quindi $x \in A \iff x \notin \overline{A}$ e, analogamente, $x \in \overline{A} \iff x \notin A$.

0.2.3 Relazioni

Per definire le relazioni è necessario il *prodotto cartesiano*. Dati gli insiemi $A_1, A_2, \ldots, A_n$, il loro prodotto cartesiano è definito come mostrato nella Figura 0.1.

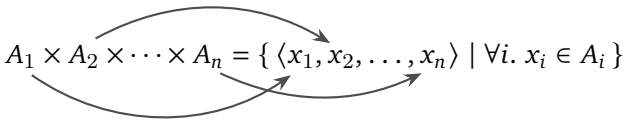


Figura 0.1 *Prodotto cartesiano: la componente x_i di ogni tupla appartiene all'insieme A_i.*

Gli elementi del prodotto cartesiano sono le *tuple* $\langle x_1, x_2, \ldots, x_n \rangle$, in cui la componente i-esima x_i appartiene all'insieme A_i. A differenza degli insiemi, nelle tuple l'ordine è rilevante e gli elementi possono ripetersi: $\langle 1, 2 \rangle \neq \langle 2, 1 \rangle$, mentre $\{1, 2\} = \{2, 1\}$.

Definizione (Relazione binaria). *Dati due insiemi A e B, una* relazione binaria *tra A e B è un sottoinsieme del loro prodotto cartesiano:*

$$R \subseteq A \times B.$$

Se $x_1 \in A$ e $x_2 \in B$ sono in relazione si scrive

$$x_1 \ R \ x_2 \qquad \textit{oppure} \qquad (x_1, x_2) \in R.$$

Le coppie, ovvero le tuple di due elementi, si indicano anche con le parentesi tonde.

0.2.4 Proprietà delle Relazioni

Le proprietà seguenti riguardano le relazioni su un insieme S, ovvero $R \subseteq S \times S$.

Definizione (Relazione riflessiva). *Una relazione R è* riflessiva *se ogni elemento è in relazione con se stesso:*

$$\forall x \in S.\ x \ R \ x.$$

Esempio. La relazione $\subseteq$ tra insiemi è riflessiva, poiché ogni insieme è sottoinsieme di se stesso:

$$\{1, 2, 5, 8\} \subseteq \{1, 2, 5, 8\}.$$

La riflessività deriva dal fatto che il simbolo $\subseteq$ ammette l'uguaglianza; il sottoinsieme proprio $\subset$, invece, non è riflessivo, poiché nessun insieme è sottoinsieme proprio di se stesso. ◇

Definizione (Relazione simmetrica). *Una relazione R è* simmetrica *se la posizione degli elementi nella relazione non è rilevante:*

$$\forall x, y \in S.\ x\ R\ y \implies y\ R\ x.$$

Esempio. L'uguaglianza tra insiemi è simmetrica: se $A = B$, allora anche $B = A$.

$$\underbrace{\{2, 5, 10\}}_{A} = \underbrace{\{2, 5, 10\}}_{B}$$

La relazione $\subseteq$, invece, non è simmetrica: $\{1\} \subseteq \{1, 2\}$, ma $\{1, 2\} \not\subseteq \{1\}$. ◇

Definizione (Relazione transitiva). *Una relazione R è* transitiva *se*

$$\forall x, y, z \in S.\ x\ R\ y \land y\ R\ z \implies x\ R\ z.$$

In altri termini, se x è in relazione con y e y è in relazione con z, allora anche x è in relazione con z, come illustrato nella Figura 0.2.

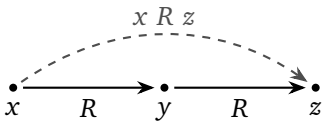


Figura 0.2 *Relazione transitiva: da $x\ R\ y$ e $y\ R\ z$ segue $x\ R\ z$.*

Definizione (Chiusura transitiva). *La* chiusura transitiva *di una relazione $R \subseteq S \times S$ è la relazione R^* definita come il più piccolo insieme $R^* \supseteq R$ tale che R^* sia transitiva: essa si ottiene aggiungendo a R le sole coppie necessarie a renderla transitiva.*

Esempio. Se $R = \{(1, 2), (2, 3)\}$, allora

$$R^* = \{(1, 2), (2, 3), (1, 3)\},$$

dove la coppia $(1, 3)$ è necessaria poiché $1\ R\ 2$ e $2\ R\ 3$. ◇

Definizione (Relazione antisimmetrica). *Una relazione R è* antisimmetrica *se*

$$\forall x, y \in S.\ x\ R\ y \land y\ R\ x \implies x = y.$$

In altri termini, elementi diversi non possono essere scambiati nella relazione: se $x \neq y$ e $x\ R\ y$, allora non può valere $y\ R\ x$.

Esempio. La relazione $\subseteq$ è antisimmetrica, poiché $A \subseteq B$ e $B \subseteq A$ implicano insieme $A = B$. ◇

0.2.5 Tipologie di Relazione

Definizione (Tipi di relazione). *I tipi classici di relazione $R \subseteq S \times S$ sono:*

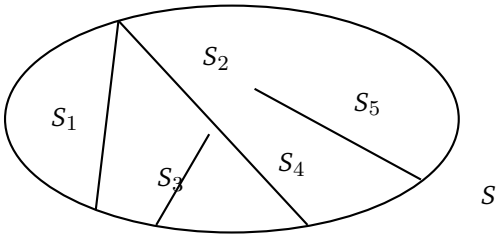
- *Equivalenza: R è riflessiva, simmetrica e transitiva.*
- *Pre-ordine: R è riflessiva e transitiva. Si tratta di un ordinamento molto debole, poiché manca l'antisimmetria: elementi diversi possono essere in relazione in entrambi i versi.*
- *Ordine parziale: R è un pre-ordine antisimmetrico.* Parziale *significa che non tutti gli elementi sono in relazione tra loro.*
- *Ordine totale: R è un ordine parziale in cui due elementi qualsiasi sono sempre in relazione:*

$$\forall x, y \in S.\ x\ R\ y \vee y\ R\ x.$$

Partizione di S

Una relazione di equivalenza $R \subseteq S \times S$ induce una **partizione** di S, ovvero una famiglia di sottoinsiemi $\{S_1, S_2, \ldots, S_m\}$ tale che:

$$\begin{cases} \bigcup_{i=1}^{m} S_i = S \\ \forall i \neq j, \quad S_i \cap S_j = \emptyset \end{cases}$$



Esempio. La relazione = è di equivalenza, $\subseteq$ è un ordine parziale ($\{1\}$ e $\{2\}$ non sono in relazione in nessuno dei due versi) e $\leq$ sui numeri naturali è un ordine totale. ◇

0.2.6 Elementi Minimali e Massimali

Nel seguito una relazione d'ordine è indicata con $\leq$. La negazione di una relazione si indica barrandone il simbolo: $y \not\leq x$ significa che $y \leq x$ non vale.

Definizione (Elementi minimali e massimali). *Un elemento $x \in S$ è* minimale *rispetto a $\leq$ in S se nessun altro elemento di S è minore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ y \not\leq x.$$

Analogamente, $x \in S$ è massimale *rispetto a $\leq$ in S se nessun altro elemento di S è maggiore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ x \not\leq y.$$

In entrambe le definizioni è necessario escludere $y = x$: poiché $\leq$ è riflessiva, vale sempre $x \leq x$, e la condizione $\forall y \in S.\ y \not\leq x$ non sarebbe mai soddisfatta.

Se l'elemento minimale è unico si chiama *minimo*; se l'elemento massimale è unico si chiama *massimo*.[1]

Esempio. In $S = \{\{1\}, \{2\}, \{1,2\}\}$ con la relazione $\subseteq$, gli elementi minimali sono $\{1\}$ e $\{2\}$, dunque non esiste un minimo, mentre l'unico elemento massimale $\{1,2\}$ è il massimo. $\diamond$

0.2.7 Funzioni

DEFINIZIONE (Funzione). *Una funzione è un caso particolare di relazione. Dati due insiemi D, il* dominio*, e C, il* codominio*, una relazione*

$$f \subseteq D \times C$$

è una funzione *da D a C, e si scrive $f\colon D \to C$, se*

$$\forall x \in D.\ \exists!\, y \in C \text{ tale che } f(x) = y.$$

Qui $f(x) = y$ significa $(x, y) \in f$ e il simbolo $\exists!$ si legge *esiste un unico*. Equivalentemente, a ogni elemento del dominio corrisponde uno e un solo elemento del codominio.

DEFINIZIONE (Immagine). *L'*immagine *di f, $\mathrm{range}(f) \subseteq C$, è l'insieme dei valori del codominio effettivamente assunti da f:*

$$\mathrm{range}(f) = \{\, c \in C \mid \exists d \in D.\ f(d) = c \,\} = \{\, f(d) \in C \mid d \in D \,\}.$$

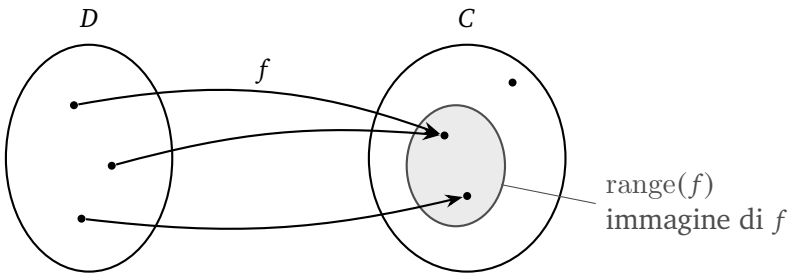


FIGURA 0.3 *Una funzione $f\colon D \to C$ e la sua immagine* $\mathrm{range}(f)$.

Nella Figura 0.3 ogni elemento di D è origine di esattamente una freccia, come richiesto dalla definizione di funzione, mentre un elemento di C può esserne destinazione più volte oppure mai; in quest'ultimo caso non appartiene all'immagine.

DEFINIZIONE (Funzione iniettiva). *Una funzione $f\colon D \to C$ è* iniettiva *se elementi diversi del dominio hanno sempre immagini diverse:*

$$\forall x, y \in D.\ f(x) = f(y) \implies x = y.$$

Se f è iniettiva, dunque, due elementi distinti del dominio non possono avere la stessa immagine

[1] Questo vale per gli insiemi finiti. In generale il minimo è l'elemento x tale che $\forall y \in S.\ x \leq y$ e il massimo è l'elemento x tale che $\forall y \in S.\ y \leq x$; in un insieme infinito può esistere un unico elemento massimale che non è il massimo.

Definizione (Funzione suriettiva). *Una funzione $f\colon D \to C$ è* suriettiva *se ogni elemento del codominio è raggiunto da almeno un elemento del dominio:*

$$\forall y \in C.\ \exists x \in D.\ f(x) = y,$$

Definizione (Funzione biettiva). *Una funzione è* biettiva*, o* biezione*, se è sia iniettiva sia suriettiva: ogni elemento del codominio è raggiunto da* esattamente un *elemento del dominio.*

Due insiemi in biezione hanno la stessa cardinalità.

Definizione (Immagine inversa). *Data $f\colon D \to C$ e un sottoinsieme $Y \subseteq C$, l'*immagine inversa *di Y è l'insieme degli elementi del dominio che f manda in Y:*

$$f^{-1}(Y) = \{\, x \in D \mid f(x) \in Y \,\}.$$

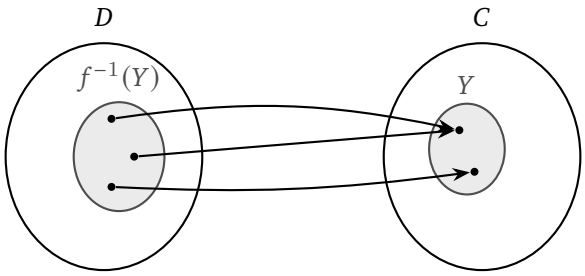


Figura 0.4 *Immagine inversa $f^{-1}(Y)$ di un sottoinsieme $Y \subseteq C$.*

1 Introduzione

1.1 Il problema dell'informatica

Si consideri una generica funzione matematica $f\colon \mathbb{N} \to \mathbb{N}$. Il problema di fondo dell'informatica consiste nello stabilire *quali funzioni siano calcolabili*.

Definizione (Funzione calcolabile).

$$f \textit{ è } \text{calcolabile} \iff \textit{esiste un algoritmo che la calcola,}$$

dove per algoritmo *si intende una sequenza finita di passi di calcolo.*

Una stessa funzione calcolabile può essere calcolata da più algoritmi diversi, come illustrato nella Figura 1.1.

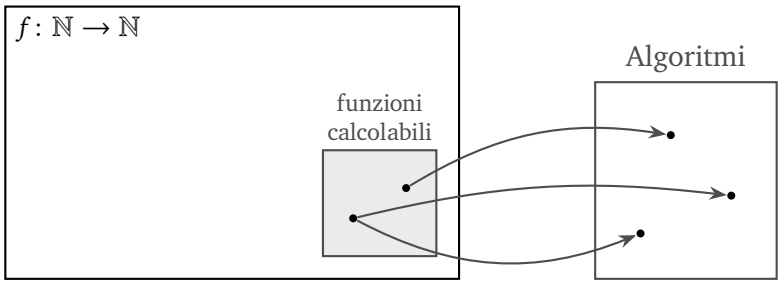


Figura 1.1 *Le funzioni calcolabili sono quelle per cui esiste almeno un algoritmo che le calcola.*

Per stabilire se tutte le funzioni siano calcolabili, si confronta la cardinalità dell'insieme delle funzioni con quella dell'insieme degli algoritmi.

1.2 Quante funzioni esistono?

Una funzione $f\colon \mathbb{N} \to \mathbb{N}$ può essere vista come un *insieme di coppie* in $\mathbb{N} \times \mathbb{N}$:

$$f(2) = 3 \mapsto (2,3) \qquad f(3) = 5 \mapsto (3,5) \qquad f(7) = 9 \mapsto (7,9)$$

Esempio. Una funzione è un insieme infinito di coppie, come

$$\{(0,3),(1,5),(2,2),(3,100),(4,1000),(5,20),(6,2),\dots\},$$

in cui ogni numero naturale compare come primo elemento di *esattamente una* coppia: non è possibile aggiungere un'altra coppia con primo elemento 2, poiché $f(2)$ è già fissato. ◇

Pertanto

$$f \subseteq \mathbb{N}\times\mathbb{N} \iff f \in \mathcal{P}(\mathbb{N}\times\mathbb{N}),$$

dove $\mathcal{P}(X)$ è l'*insieme delle parti* di X, ovvero l'insieme di tutti i suoi sottoinsiemi. Le funzioni da $\mathbb{N}$ in $\mathbb{N}$ sono quindi elementi di $\mathcal{P}(\mathbb{N}\times\mathbb{N})$.

1.2.1 $\mathbb{N}\times\mathbb{N}$ è numerabile

Vale $|\mathbb{N}\times\mathbb{N}| = |\mathbb{N}|$. Le seguenti affermazioni sono equivalenti:

- la cardinalità di $\mathbb{N}\times\mathbb{N}$ è uguale a quella di $\mathbb{N}$;
- l'insieme $\mathbb{N}\times\mathbb{N}$ è *numerabile*;
- esiste una *biezione* (mappatura univoca) tra $\mathbb{N}\times\mathbb{N}$ e $\mathbb{N}$.

Infatti, le coppie possono essere ordinate e numerate percorrendole per diagonali[1] (Figura 1.2):

$$(0,0)\mapsto 0 \quad (0,1)\mapsto 1 \quad (1,0)\mapsto 2 \quad (0,2)\mapsto 3 \quad (1,1)\mapsto 4 \quad (2,0)\mapsto 5 \quad \dots$$

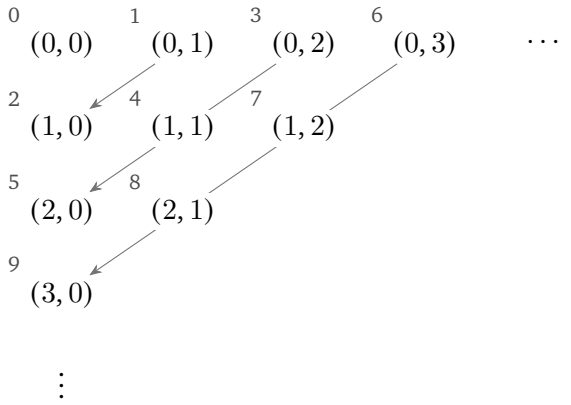


Figura 1.2 *Enumerazione delle coppie di $\mathbb{N}\times\mathbb{N}$ per diagonali.*

Poiché $\mathbb{N}\times\mathbb{N}$ e $\mathbb{N}$ sono in biezione, lo sono anche i loro insiemi delle parti. Quindi

$$|\{f \mid f\colon \mathbb{N}\to\mathbb{N}\}| = |\mathcal{P}(\mathbb{N}\times\mathbb{N})| = |\mathcal{P}(\mathbb{N})|,$$

ovvero esistono *tante funzioni quanti sono i sottoinsiemi di* $\mathbb{N}$.

[1] Non è invece possibile numerare le coppie riga per riga, $(0,0)\mapsto 0$, $(0,1)\mapsto 1$, $(0,2)\mapsto 2$, …: la riga delle coppie che iniziano con 0 è infinita, quindi la coppia $(1,0)$ non riceverebbe mai un numero.

1.3 Quanti algoritmi esistono?

Sia Σ un *alfabeto*, ovvero un insieme finito e non vuoto di simboli: quelli utilizzati per scrivere programmi e algoritmi. Un *programma* è una *stringa* (sequenza di simboli) su Σ, quindi

$$|\text{Programmi}| = |\text{stringhe su } \Sigma| = |\Sigma^*|,$$

Dove * è la *chiusura di Kleene*, ovvero l'insieme di tutte le stringhe ottenute combinando 0 o più volte gli elementi di Σ.
Σ^* è, invece, l'insieme delle stringhe *finite* su Σ, ad esempio:

$$\underbrace{\Sigma = \{a, b\}}_{\text{finito}} \qquad \underbrace{\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}}_{\text{infinito}}$$

dove ε è la *stringa vuota*.

Σ^* può essere messo in biezione con $\mathbb{N}$ poiché i suoi elementi sono ordinabili (prima per lunghezza, poi in ordine alfabetico, come nell'esempio): è sufficiente numerarli secondo tale ordine. Quindi Σ^* è numerabile, $|\Sigma^*| = |\mathbb{N}|$, e

$$|\text{Programmi}| = |\Sigma^*| = |\mathbb{N}|.$$

1.4 Il teorema di Cantor

Resta da confrontare $|\mathbb{N}|$, la cardinalità degli algoritmi, con $|\mathcal{P}(\mathbb{N})|$, la cardinalità delle funzioni.

Teorema 1 (di Cantor). *Per ogni insieme S si ha che*

$$|S| < |\mathcal{P}(S)|.$$

Per gli insiemi finiti il teorema è verificabile direttamente, come mostrato nella Tabella 1.1.

Insieme S	Insieme delle parti $\mathcal{P}(S)$	$\lvert S\rvert$	$\lvert\mathcal{P}(S)\rvert$
$\emptyset$	$\{\emptyset\}$	0	1
$\{a, b\}$	$\{\emptyset, \{a\}, \{b\}, \{a, b\}\}$	2	4
$\{a, b, c\}$	$\{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}\}$	3	8

Tabella 1.1 *Cardinalità di alcuni insiemi finiti e dei loro insiemi delle parti.*

Dimostrazione del Teorema 1. Si suppone per assurdo che $|S| = |\mathcal{P}(S)|$. Allora esiste una funzione biunivoca $f : S \to \mathcal{P}(S)$ che mappa ogni elemento di S in un sottoinsieme di S stesso.

Si consideri l'insieme $A = \{x \in S \mid x \notin f(x)\} \subseteq S$. Dato che $\mathcal{P}(S)$ contiene ogni sottoinsieme di S e A ne è uno, allora $A \in \mathcal{P}(S)$. Pertanto, obbligatoriamente
$\exists a \in S \,.\, f(a) = A$. Quindi si hanno due possibilità:

- $a \in A$: allora, per costruzione di A, $a \notin f(a) = A$, $\lightning$.
- $a \notin A$: simmetricamente, per definizione di A, $a \in f(a) = A$, $\lightning$.

Poiché in entrambi i casi si raggiunge una contraddizione, allora $\nexists a \in S \,.\, f(a) = A$. Si conclude, quindi, che A è un elemento privo di controimmagine e, di conseguenza, la cardinalità di S è strettamente inferiore a quella di $\mathcal{P}(S)$. □

In generale

$$|\mathcal{P}(S)| = 2^{|S|}.$$

Per $\mathbb{N}$ vale $|\mathcal{P}(\mathbb{N})| = |\mathbb{R}|$: i sottoinsiemi di $\mathbb{N}$ sono tanti quanti i numeri reali, che *non sono numerabili*. L'*ipotesi del continuo* afferma inoltre che non esiste alcun insieme con cardinalità strettamente compresa tra $|\mathbb{N}|$ e $|\mathbb{R}|$.

Riassumendo:

$$|\text{Programmi}| = |\mathbb{N}| < |\mathcal{P}(\mathbb{N})| = |\{f \mid f \colon \mathbb{N} \to \mathbb{N}\}|.$$

Poiché la cardinalità dei programmi è strettamente inferiore a quella delle funzioni, esistono *funzioni non calcolabili*.

2 Induzione

2.1 Relazioni ben fondate

Definizione (Relazione ben fondata). *Sia A un insieme e $\prec \subseteq A \times A$ una relazione su A, antisimmetrica e transitiva. La relazione $\prec$ è* ben fondata *se non esistono* catene discendenti infinite*, ovvero sequenze infinite $a_0 \succ a_1 \succ a_2 \succ \ldots$ di elementi di A.*

Ne segue che, se A non è vuoto, esistono *elementi minimali*, ovvero elementi di A privi di elementi più piccoli. L'insieme degli elementi minimali di A si indica con $\mathrm{Base}(A)$:

$$\mathrm{Base}(A) = \{\, a \in A \mid \nexists b \in A.\ b \prec a \,\}.$$

La situazione è illustrata nella Figura 2.1.

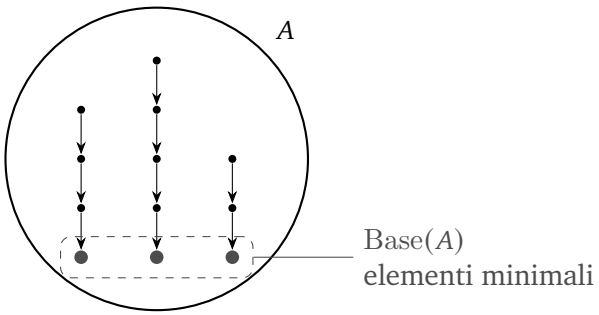


Figura 2.1 *Catene discendenti in un insieme ben fondato e suoi elementi minimali.*

2.2 Principio di induzione

Teorema 2 (Principio di induzione). *Sia $(A, \prec)$ ben fondato e sia Π una proprietà sugli elementi di A. Allora $\forall a \in A.\ \Pi(a)$ se e solo se valgono:*

1. Base dell'induzione*: $\forall a \in \mathrm{Base}(A).\ \Pi(a)$;*
2. Passo induttivo*: per ogni $a \in A \setminus \mathrm{Base}(A)$*

$$\underbrace{\big(\forall b \prec a.\ \Pi(b)\big)}_{\textit{ipotesi induttiva}} \implies \underbrace{\Pi(a)}_{\textit{tesi}}$$

Esempio. Si vuole dimostrare che

$$\forall n \geq 1. \quad \sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Sia $\Pi(n)$ tale uguaglianza. In questo caso $A = \mathbb{N} \setminus \{0\}$ con l'ordine usuale $<$, che è ben fondato, e $\mathrm{Base}(A) = \{1\}$.

Base. Per $n = 1$, $\Pi(1)$ diventa:

$$\sum_{i=1}^{1} i = 1 \qquad \frac{1 \cdot (1+1)}{2} = \frac{1 \cdot 2}{2} = 1.$$

I due membri coincidono, dunque $\Pi(1)$ è verificata.

Passo induttivo. Sia $n \in \mathbb{N} \setminus \{0, 1\}$ e si assuma l'*ipotesi induttiva*, ovvero che $\Pi(m)$ valga per ogni $m < n$. In particolare vale $\Pi(n-1)$, utilizzata nel secondo passaggio:

$$\begin{aligned}
\sum_{i=1}^{n} i &= \sum_{i=1}^{n-1} i + n \\
&= \frac{(n-1)(n-1+1)}{2} + n && \text{(ipotesi induttiva)} \\
&= \frac{(n-1)n + 2n}{2} \\
&= \frac{n(n-1+2)}{2} \\
&= \frac{n(n+1)}{2},
\end{aligned}$$

che è esattamente $\Pi(n)$. ◇

3 Linguaggi Formali

3.1 Alfabeti e stringhe

Definizione (Alfabeto). *Un* alfabeto Σ *è un insieme finito e non vuoto di simboli, ovvero di entità primitive come lettere e caratteri.*

Definizione (Stringa). *Una* stringa*, o* parola*, è una sequenza finita di simboli su un alfabeto* Σ*.*

Come visto nella Sezione 1.3, l'insieme di tutte le stringhe su Σ si indica con Σ^*. Esso contiene sempre la *stringa vuota* ε, priva di simboli, ed è infinito numerabile.

Definizione (Lunghezza di una stringa). *La* lunghezza $|x|$ *di una stringa* x *è il numero di* occorrenze *di simboli al suo interno.*

Si contano le occorrenze e non i simboli: ad esempio $|abba| = 4$, anche se $abba$ contiene solo due simboli distinti. La stringa vuota ha lunghezza $|\varepsilon| = 0$.

Definizione (Prefisso, suffisso e sottostringa). *Sia* $x = a_1 a_2 \cdots a_n$ *una stringa su* Σ*, con* $a_1, \ldots, a_n \in \Sigma$*, e siano* $1 \leq i \leq j \leq n$*. Allora:*

- $a_1 \cdots a_i$*, la parte iniziale di* x*, è un* prefisso *di* x*;*
- $a_j \cdots a_n$*, la parte finale di* x*, è un* suffisso *di* x*;*
- $a_i \cdots a_j$*, una qualsiasi parte contigua di* x*, è una* sottostringa *di* x*.*

Anche la stringa vuota ε è prefisso, suffisso e sottostringa di ogni stringa. Come mostra la Figura 3.1, ogni prefisso e ogni suffisso di x sono anche sue sottostringhe.

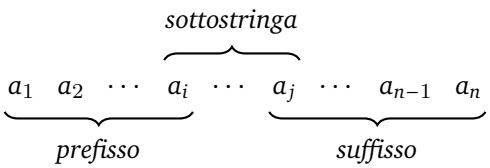


Figura 3.1 *Prefisso* $a_1 \cdots a_i$*, sottostringa* $a_i \cdots a_j$ *e suffisso* $a_j \cdots a_n$ *della stringa* $x = a_1 a_2 \cdots a_n$*.*

3.2 Linguaggi

Definizione (Linguaggio). *Un* linguaggio $\mathcal{L}$ *su un alfabeto* Σ *è un sottoinsieme delle stringhe di* Σ^**, ovvero* $\mathcal{L} \subseteq \Sigma^*$.

Alcuni esempi di linguaggi sono $\emptyset$, dato che è sottoinsieme di qualsiasi insieme, e $\{\varepsilon\}$, il linguaggio che contiene solo la stringa vuota. I due non vanno confusi: $\emptyset$ non contiene alcuna stringa, mentre $\{\varepsilon\}$ ne contiene esattamente una, di lunghezza 0. Entrambi sono linguaggi su qualsiasi alfabeto, mentre Σ^* è il linguaggio di tutte le stringhe su Σ.

Poiché un linguaggio è, di fatto, un insieme, su di esso si possono eseguire le operazioni insiemistiche della Sezione 0.2.2, con Σ^* come insieme universo:

- Complemento di $\mathcal{L} \subseteq \Sigma^*$: $\overline{\mathcal{L}} = \neg\mathcal{L} = \Sigma^* \setminus \mathcal{L}$;
- Unione di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cup \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \vee x \in \mathcal{L}_2\}$;
- Intersezione di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cap \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \wedge x \in \mathcal{L}_2\}$.

Il complemento è quindi sempre relativo a Σ^*. Ad esempio, il complemento del linguaggio vuoto è il linguaggio di tutte le stringhe, mentre quello di $\{\varepsilon\}$ è il linguaggio delle stringhe non vuote:

$$\overline{\emptyset} = \Sigma^* \setminus \emptyset = \Sigma^*, \qquad \overline{\{\varepsilon\}} = \{x \in \Sigma^* \mid x \neq \varepsilon\} = \Sigma^* \setminus \{\varepsilon\}.$$

Altre operazioni invece sono esclusivamente legate ai linguaggi:

- Concatenazione: $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^* : \mathcal{L}_1 \cdot \mathcal{L}_2 = \{\ xy\ \in \Sigma^* \mid\ x \in \mathcal{L}_1, y \in \mathcal{L}_2\}$
- Stella di Kleene: $\mathcal{L} \subseteq \Sigma^*$:

$$\mathcal{L}^n = \begin{cases} \mathcal{L}^0 = \{\epsilon\} \\ \mathcal{L}^{n+1} = \mathcal{L} \cdot \mathcal{L}^n \end{cases} \qquad \mathcal{L}^* = \bigcup_{n \in \mathbb{N}} \mathcal{L}^n$$

3.3 Funzioni come linguaggi

Una funzione $f : \mathbb{N} \to \mathbb{N}$ può essere vista come un linguaggio $\mathcal{L}_f$ sull'alfabeto Σ composto da un singolo carattere.

Esempio. Si consideri l'alfabeto $\Sigma = \{1\}$ e una funzione $f : \mathbb{N} \to \mathbb{N}$. Allora:

$$f : \mathbb{N} \to \mathbb{N} \quad \longleftrightarrow \quad \mathcal{L}_f = \{1^{f(x)} \mid x \in \mathbb{N}\},$$

dove 1^n indica la stringa formata da n occorrenze del simbolo 1, con $1^0 = \varepsilon$. ◇

Questo linguaggio permette di porre una domanda con risposta sì o no: data in input una stringa $\sigma \in \Sigma^*$, stabilire se essa appartenga o meno al linguaggio,

$$\sigma \in \Sigma^* \ \longmapsto \ \begin{cases} \sigma \in \mathcal{L}_f & \text{se } \sigma \text{ appartiene al linguaggio,} \\ \sigma \notin \mathcal{L}_f & \text{se } \sigma \text{ non appartiene al linguaggio.} \end{cases}$$

Si ragiona così su insiemi invece che su funzioni, dove gli elementi dell'insieme dipendono dal calcolo della funzione.

Esempio. Per $f(x) = 2x$ si ottiene

$$\mathcal{L}_f = \{1^{2x} \mid x \in \mathbb{N}\} = \{\varepsilon, 11, 1111, 111111, \dots\},$$

il linguaggio delle stringhe di lunghezza pari su $\{1\}$: ad esempio $1111 \in \mathcal{L}_f$, poiché $f(2) = 4$, mentre $111 \notin \mathcal{L}_f$, poiché nessun x ha $f(x) = 3$. ◇

3.4 Automi a stati finiti deterministici

I linguaggi regolari sono tutti quei linguaggi per cui esiste un automa a stati finiti deterministico che lo riconosce.

Un automa a stati finiti è un modello matematico di un sistema avente input, ed eventualmente output, a valori discreti. Il sistema può essere in uno stato tra un insieme finito di stati possibili. L'essere in uno stato gli permette di tener traccia della storia precedente.

Un automa a stati finiti è rappresentabile da una testina che legge, spostandosi sempre nella stessa direzione e un nastro di lunghezza illimitata contenente dei simboli. La testina può puntare a un certo *stato* q e, a seconda del simbolo w_i, la testina punterà ad un altro stato (oppure sempre allo stesso). La situazione è illustrata nella Figura 3.2.

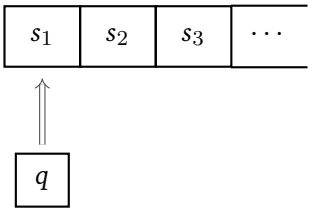


Figura 3.2 *Automa a stati finiti: la testina, nello stato q, legge il primo simbolo s_1 del nastro.*

Quando la lettura dei simboli termina, a seconda dello stato raggiunto dalla testina, l'automa fornisce un risultato di accettazione o di rifiuto della stringa letta.

Il comportamento dell'automa si definisce in maniera univoca mediante una tabella detta *matrice di transizione*.

Esempio. Matrice di Transizione

	a	b
w_0	w_1	w_2
w_1	w_1	w_0
w_2	w_1	w_0

◇

Ma è anche comune la rappresentazione grafica con grafi.

Definizione (Automa Deterministico). *Un automa a stati finiti deterministico (DFA) è una quintupla del tipo $\langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ dove:*

- Q *è un insieme finito di stati;*
- Σ *è un alfabeto;*
- $\delta : Q \times \Sigma \to Q$ *è la* funzione di transizione*;*
- q_0 *è lo stato* iniziale*;*

- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali

I DFA hanno una funzione di transizione deterministica. L'unica forma di memoria è lo stato in cui l'automa si trova. Inoltre:

$\forall Stato \forall Simbolo$ esiste sempre una transizione

Notazione: Useremo p, q, r con o senza pedici per denotare stati, P, Q, R, S per insiemi di stati, a, b con o senza pedici per denotare simboli di Σ, x, y, z, u, v, w sempre con o senza pedici per denotare stringhe.

Poiché la funzione δ acquisisce in ingresso un solo simbolo, è necessario estenderla per poter leggere stringhe. Questo viene eseguito mediante la funzione $\hat{\delta}$, che si ottiene in modo univoco da δ ed è definita come segue:

$$\hat{\delta}\colon Q \times \Sigma^* \to Q, \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = q, \\ \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a). \end{cases}$$

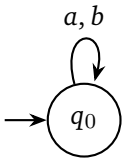
Una stringa x è detta *accettata* o riconosciuta da un DFA $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ se $\hat{\delta}(q_0, x) \in \mathcal{F}$. Il linguaggio riconosciuto/accettato da $\mathcal{M}$, denotato come $\mathcal{L}(\mathcal{M})$ è l'insieme delle stringhe accettate, ovvero:

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* : \hat{\delta}(q_0, x) \in \mathcal{F}\}$$

Definizione (Linguaggio regolare). *Un linguaggio $\mathcal{L}$ è detto* regolare *se è accettato da qualche DFA, ovvero se esiste un DFA $\mathcal{M}$ tale che*

$$\mathcal{L} = \mathcal{L}(\mathcal{M}).$$

Esempio. Il linguaggio vuoto $\mathcal{L} = \emptyset$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.3, con $Q = \{q_0\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \emptyset$.



δ	a	b
q_0	q_0	q_0

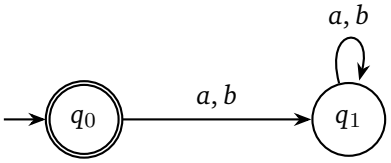
Figura 3.3 *DFA che riconosce il linguaggio vuoto e la sua matrice di transizione.*

Qualunque simbolo legga, l'automa resta in q_0, quindi $\hat{\delta}(q_0, x) = q_0$ per ogni $x \in \Sigma^*$. Poiché $\mathcal{F} = \emptyset$, però, la condizione di accettazione $\hat{\delta}(q_0, x) \in \mathcal{F}$ è sempre falsa: nessuno stato appartiene all'insieme vuoto. Nessuna stringa viene quindi accettata e

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \underbrace{\hat{\delta}(q_0, x) \in \mathcal{F}}_{\text{sempre falsa, poiché } \mathcal{F} = \emptyset}\} = \emptyset.$$

Riconoscere $\emptyset$ non significa che l'automa non esista, ma che non accetta alcuna stringa: in generale, ogni DFA con $\mathcal{F} = \emptyset$ riconosce il linguaggio vuoto. ◇

Esempio. Il linguaggio $\mathcal{L} = \{\varepsilon\}$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.4, con $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \{q_0\}$.



δ	a	b
q_0	q_1	q_1
q_1	q_1	q_1

Figura 3.4 *DFA che riconosce il linguaggio $\{\varepsilon\}$ e la sua matrice di transizione: lo stato finale q_0 è indicato dal doppio cerchio.*

Lo stato iniziale è anche finale, quindi la stringa vuota, che non fa compiere alcuna transizione, viene accettata. Il primo simbolo letto porta invece in q_1, da cui l'automa non esce più:

$$\hat{\delta}(q_0, \varepsilon) = q_0 \in \mathcal{F}, \qquad \hat{\delta}(q_0, x) = q_1 \notin \mathcal{F} \quad \text{per ogni } x \neq \varepsilon.$$

Quindi $\mathcal{L}(\mathcal{M}) = \{\varepsilon\}$. ◇

3.4.1 Come dimostrare che un linguaggio è regolare

Un modo per dimostrare che un linguaggio è regolare è costruire un DFA che lo riconosca. L'uguaglianza $\mathcal{L} = \mathcal{L}(\mathcal{M})$ verrà dimostrata mediante la duplice inclusione $\mathcal{L} \subseteq \mathcal{L}(\mathcal{M})$ e $\mathcal{L}(\mathcal{M}) \subseteq \mathcal{L}$. In particolare:

$$\begin{aligned}
& \mathcal{L}(\mathcal{M}) \subseteq \mathcal{L} \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L}(\mathcal{M}) \implies x \in \mathcal{L}\big) \\
\iff & \hat{\delta}(q_0, x) \in \mathcal{F} \implies x \in \mathcal{L} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)} \\
\iff & x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F} && \text{(per contrapposizione)}
\end{aligned}$$

e

$$\begin{aligned}
& \mathcal{L} \subseteq \mathcal{L}(\mathcal{M}) \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L} \implies x \in \mathcal{L}(\mathcal{M})\big) \\
\iff & x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)}
\end{aligned}$$

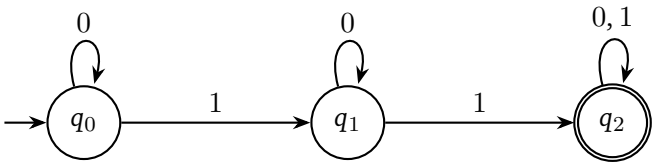
Riassumendo:

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F}, \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F}. \end{cases}$$

Dal momento che le condizioni da dimostrare sono analoghe, verranno portate avanti insieme ed è quindi necessario che la base dell'induzione sia la stessa per entrambe. In particolare, è richiesto di trovare una lunghezza tale per cui esistano stringhe in $\mathcal{L}$ e stringhe non in $\mathcal{L}$. I passi sono dunque:

- Base: si considera la lunghezza minima di x e, per tutte le lunghezze inferiori o uguali a quella, si dimostra che valgono entrambe le condizioni;
- Passo induttivo: si suppone che entrambe le condizioni valgano per ogni x tale che $|x| \leq n$ e si dimostra che valgono anche per $|x| = n + 1$.

Esempio. Si consideri il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid \text{“}x \text{ contiene almeno due } 1\text{”}\}$ definito sull'alfabeto binario $\Sigma = \{0, 1\}$. L'automa a stati finiti che riconosce $\mathcal{L}$ è quello della Figura 3.5, con stato iniziale q_0 e $\mathcal{F} = \{q_2\}$.



δ	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

FIGURA 3.5 *DFA che riconosce le stringhe binarie con almeno due* 1 *e la sua matrice di transizione.*

È comunque necessario dimostrare che vengono riconosciute tutte e sole le stringhe di $\mathcal{L}$. In particolare, bisogna dimostrare che

$$\mathcal{L} = \mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \in \{q_2\}\}.$$

La dimostrazione avviene per induzione. La base non può riguardare *solo* stringhe con meno di due simboli: servono almeno due simboli per comporre una stringa $z \in \mathcal{L}$.[1] Quindi:

$$\forall x \in \Sigma^*.\ \big(|x| < 2 \implies x \notin \mathcal{L}\big)$$

Allora:

$$\left.\begin{array}{lll}
|x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} \\
|x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 \notin \mathcal{F} \\
 & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 \notin \mathcal{F} \\
|x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 \notin \mathcal{F} \\
 & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 \notin \mathcal{F} \\
 & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 \notin \mathcal{F} \\
 & x = 11 \in \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 \in \mathcal{F}
\end{array}\right\} \text{ BASE}$$

PASSO INDUTTIVO. *Ipotesi induttiva*: si suppone che per tutte le stringhe x, con $|x| \geq 2$, di lunghezza $|x| \leq n$, per un certo $n \in \mathbb{N}$, valga che

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F} \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F} \end{cases}$$

dove si è usato che $\mathcal{F} = \{q_2\}$. La seconda condizione, però, non basta: dice solo che l'automa è in q_0 oppure in q_1, mentre per leggere il simbolo successivo serve sapere in quale dei due. La si scorpora quindi in due condizioni più precise, ottenendo l'ipotesi

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, \\ x \notin \mathcal{L} \wedge \text{“}x \text{ non contiene } 1\text{”} \implies \hat{\delta}(q_0, x) = q_0, \\ x \notin \mathcal{L} \wedge \text{“}x \text{ contiene un solo } 1\text{”} \implies \hat{\delta}(q_0, x) = q_1. \end{cases}$$

[1] In realtà, non è mai possibile considerare $|x| = 0$ come unico caso base, perché l'unica stringa di tale lunghezza, ε, o appartiene a un linguaggio o non vi appartiene. Servono, invece, lunghezze tali per cui esistono stringhe sia appartenenti che non.

È lecito farlo perché ogni nuova condizione implica quella di partenza, dato che q_0 e q_1 sono diversi da q_2, e la base verifica già gli stati esatti. In cambio, nel passo induttivo va dimostrata per x l'intera $\Psi(x)$.

Si dimostri allora il caso base per Ψ:

$$\left.\begin{array}{lllll}
|x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 & \text{(caso 2 di } \Psi) & \\
|x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 & \text{(caso 2 di } \Psi) & \\
 & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 & \text{(caso 3 di } \Psi) & \\
|x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 & \text{(caso 2 di } \Psi) & \\
 & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 & \text{(caso 3 di } \Psi) & \\
 & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 3 di } \Psi) & \\
 & x = 11 \in \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 & \text{(caso 1 di } \Psi) &
\end{array}\right\} \text{Base}$$

Passo induttivo. Affinché $\Psi(x)$ possa essere verificata per tutte le stringhe $\in \Sigma^*$, è necessario applicare l'ipotesi induttiva:

$$\forall x \in \Sigma^*.\ \big(|x| \leq n \implies \Psi(x)\big)$$

Sia x una stringa con $|x| = n + 1$: allora $x = x'0$ oppure $x = x'1$, con $|x'| = n$, e a x' si può applicare l'ipotesi induttiva. Si distingue un caso per ciascuna condizione di $\Psi(x)$. In ogni caso si ragiona prima sul linguaggio, contando gli 1 di x'; poi si applica l'ipotesi induttiva a x' e infine si legge l'ultimo simbolo. Nei calcoli, *def* indica la definizione di $\hat{\delta}$ e *Hp* il punto in cui si sostituisce lo stato dato dall'ipotesi induttiva.

1. $x \in \mathcal{L}$, quindi x contiene almeno due 1.

   1a. $x = x'0$. Se x' contenesse un solo 1 (o nessuno), anche $x = x'0$ ne conterrebbe meno di due e quindi $x \notin \mathcal{L}$: assurdo. Quindi x' contiene almeno due 1, cioè $x' \in \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_2$. Allora

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_2, 0) = q_2 \in \mathcal{F}.$$

   1b. $x = x'1$. Se x' non contenesse 1, $x = x'1$ ne conterrebbe uno solo e quindi $x \notin \mathcal{L}$: assurdo. Quindi x' contiene almeno due 1, e quindi $x' \in \mathcal{L}$, oppure ne contiene esattamente uno, e quindi $x' \notin \mathcal{L}$.
   Se $x' \in \mathcal{L}$, per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$, quindi

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_2, 1) = q_2 \in \mathcal{F}.$$

   Se $x' \notin \mathcal{L}$, x' contiene un solo 1 e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_1$: è qui che serve lo scorporo, perché altrimenti si saprebbe solo che $\hat{\delta}(q_0, x') \neq q_2$. Quindi

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_1, 1) = q_2 \in \mathcal{F}.$$

2. $x \notin \mathcal{L}$ e x non contiene 1. Il caso $x = x'1$ è assurdo, perché x conterrebbe un 1, quindi $x = x'0$. Se x' contenesse almeno un 1, lo conterrebbe anche x: assurdo. Perciò x' non contiene 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_0$. Allora

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_0, 0) = q_0.$$

3. $x \notin \mathcal{L}$ e x contiene un solo 1.

   3a. $x = x'1$. Se x' contenesse almeno un 1, $x = x'1$ ne conterrebbe almeno due e quindi $x \in \mathcal{L}$: assurdo. Perciò x' non contiene 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_0$. Allora

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_0, 1) = q_1.$$

   3b. $x = x'0$. Se x' non contenesse 1, nemmeno $x = x'0$ ne conterrebbe; se ne contenesse almeno due, $x \in \mathcal{L}$. In entrambi i casi è assurdo, perciò x' contiene un solo 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_1$. Allora

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_1, 0) = q_1.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale anche per $|x| = n + 1$. Per induzione vale per ogni $x \in \Sigma^*$: in particolare $\hat{\delta}(q_0, x) \in \mathcal{F}$ se e solo se $x \in \mathcal{L}$, ovvero $\mathcal{L}(\mathcal{M}) = \mathcal{L}$. ◇

3.5 Automi a stati finiti non deterministici

In un Automa a stati Finiti non Deterministico (NFA), la funzione di transizione è l'unico elemento che varia rispetto ai Deterministici. L'obiettivo risulta verificare se il non determinismo cambia la potenza di calcolo.

I NFA hanno una funzione di transizione non deterministica. L'unica forma di memoria è lo stato in cui l'automa si trova. Inoltre:

$\forall Stato \forall Simbolo$ posso eseguire zero o più transizioni di stato

Non c'è l'obbligo di cambiare stato ad ogni transizione. È dunque possibile leggere simboli senza cambiare stato.

Definizione (Automa Non Deterministico). *Un automa a stati finiti non deterministico (NFA) è una quintupla del tipo $\langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ dove:*

- Q *è un insieme finito di stati;*
- Σ *è un alfabeto;*
- $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ *è la* funzione di transizione*: per ogni input posso raggiungere più stati. Pertanto è una funzione non deterministica;*
- q_0 *è lo stato* iniziale*;*
- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali

Poiché la funzione δ acquisisce in ingresso un solo simbolo, è necessario estenderla per poter leggere stringhe. Questo viene eseguito mediante la funzione $\hat{\delta}$, che si ottiene in modo univoco da δ ed è definita come segue:

$$\hat{\delta} \colon Q \times \Sigma^* \to \mathcal{P}(Q), \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = \{q\}, \\ \hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a) \end{cases}$$

Ad esempio, si supponga che, leggendo w a partire da q, l'automa possa arrivare negli stati p_1, p_2 e p_3, ovvero $\hat{\delta}(q, w) = \{p_1, p_2, p_3\}$. Leggendo poi il simbolo a, da ciascuno di

essi si raggiunge l'insieme $\delta(p_i, a)$, e $\hat{\delta}(q, wa)$ è l'unione di questi insiemi, come mostrato nella Figura 3.6:

$$\hat{\delta}(q, wa) = \delta(p_1, a) \cup \delta(p_2, a) \cup \delta(p_3, a).$$

Nella figura $\delta(p_1, a) = \{r_1, r_2, r_3\}$, $\delta(p_2, a) = \{r_4, r_5, r_6\}$ e $\delta(p_3, a) = \{r_7, r_8\}$, quindi

$$\hat{\delta}(q, wa) = \{r_1, r_2, r_3\} \cup \{r_4, r_5, r_6\} \cup \{r_7, r_8\} = \{r_1, \ldots, r_8\}.$$

In generale gli insiemi $\delta(p_i, a)$ possono avere stati in comune, che nell'unione compaiono una volta sola. Leggendo wa un simbolo alla volta, l'automa attraversa quindi gli insiemi di stati

$$\hat{\delta}(q, \varepsilon) = \{q\} \xrightarrow{w} \hat{\delta}(q, w) = \{p_1, p_2, p_3\} \xrightarrow{a} \hat{\delta}(q, wa) = \{r_1, \ldots, r_8\}.$$

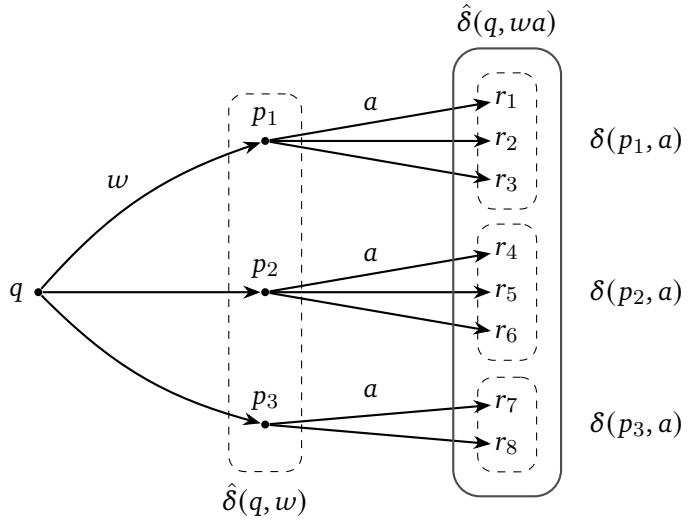


Figura 3.6 *Calcolo di $\hat{\delta}(q, wa)$ in un NFA: l'unione degli insiemi $\delta(p_i, a)$, per ogni $p_i \in \hat{\delta}(q, w)$.*

Il linguaggio accettato da un NFA $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ è l'insieme delle stringhe per cui almeno uno degli stati raggiunti è finale:

$$\mathcal{L}(\mathcal{N}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \cap \mathcal{F} \neq \emptyset\}.$$

Teorema 3 (Rabin-Scott, 1959).

a. Sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un DFA. Allora esiste un NFA $\mathcal{N} = \langle Q', \Sigma, \delta', q_0, \mathcal{F} \rangle$ tale che i due linguaggi riconosciuti coincidono, ovvero $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{N})$.

b. Sia $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un NFA. Allora esiste un DFA $\mathcal{M} = \langle Q', \Sigma, \delta', q_0', \mathcal{F}' \rangle$ tale che i due linguaggi riconosciuti coincidono, ovvero $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{M})$.

Dimostrazione del Teorema 3. L'obiettivo è dimostrare che NFA e DFA sono equivalenti.

a) Un DFA è un caso particolare di NFA, in cui ogni transizione porta in un solo stato. Si definisce quindi $\mathcal{N}$ con $Q' = Q$, gli stessi q_0 e $\mathcal{F}$, e

$$\delta'(q, a) = \{\delta(q, a)\}.$$

Per induzione su $|x|$ si verifica che $\hat{\delta}'(q_0, x) = \{\hat{\delta}(q_0, x)\}$. Per $x = \varepsilon$ entrambi i membri valgono $\{q_0\}$; per $x = wa$, con $|w| = n$, l'ipotesi induttiva dice che $\hat{\delta}'(q_0, w) = \{\hat{\delta}(q_0, w)\}$,

quindi

$$\begin{aligned}
\hat{\delta}'(q_0, wa) &= \bigcup_{p \in \hat{\delta}'(q_0,w)} \delta'(p,a) && \text{(definizione di } \hat{\delta}' \text{ per gli NFA)} \\
&= \bigcup_{p \in \{\hat{\delta}(q_0,w)\}} \delta'(p,a) && \text{(ipotesi induttiva)} \\
&= \delta'(\hat{\delta}(q_0,w), a) && \text{(unione di un solo stato)} \\
&= \{\delta(\hat{\delta}(q_0,w), a)\} && \text{(definzione di } \delta') \\
&= \{\hat{\delta}(q_0, wa)\} && \text{(definzione di } \hat{\delta})
\end{aligned}$$

Quindi $\hat{\delta}'(q_0, x) \cap \mathcal{F} \neq \emptyset$ se e solo se $\hat{\delta}(q_0, x) \in \mathcal{F}$, ovvero $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{M})$.

b) L'idea è che ogni stato di $\mathcal{M}$ rappresenti l'insieme degli stati in cui $\mathcal{N}$ può trovarsi dopo aver letto una stringa. Si definisce quindi $\mathcal{M}$ come segue:

- $Q' = \mathcal{P}(Q)$: gli stati di $\mathcal{M}$ sono i sottoinsiemi di Q, che sono $2^{|Q|}$ e quindi in numero finito[2];
- $q'_0 = \{q_0\}$: all'inizio $\mathcal{N}$ si trova solo in q_0;
- $\delta' : Q' \times \Sigma \to Q'$, con $\delta'(S, a) = \bigcup_{p \in S} \delta(p, a)$ per ogni $S \in Q'$ e $a \in \Sigma$: da S, leggendo a, si raggiungono tutti gli stati raggiungibili da almeno uno degli stati di S. Il risultato è un sottoinsieme di Q, cioè un solo stato di Q', quindi $\mathcal{M}$ è deterministico;
- $\mathcal{F}' = \{S \in Q' \mid S \cap \mathcal{F} \neq \emptyset\}$: un insieme di stati è finale se contiene almeno uno stato finale di $\mathcal{N}$.

Si dimostra per induzione su $|x|$ che, per ogni $x \in \Sigma^*$,

$$\hat{\delta}'(q'_0, x) = \hat{\delta}(q_0, x),$$

cioè che dopo aver letto x l'automa $\mathcal{M}$ si trova nello stato che corrisponde esattamente all'insieme degli stati raggiunti da $\mathcal{N}$.

Base. Per $|x| = 0$ si ha $x = \varepsilon$ e

$$\hat{\delta}'(q'_0, \varepsilon) = q'_0 = \{q_0\} = \hat{\delta}(q_0, \varepsilon).$$

Passo induttivo. Si suppone che l'uguaglianza valga per tutte le stringhe di lunghezza n e sia $x = wa$, con $|w| = n$ e $a \in \Sigma$. Allora

$$\begin{aligned}
\hat{\delta}'(q'_0, wa) &= \delta'(\hat{\delta}'(q'_0, w), a) && \text{(definizione di } \hat{\delta}') \\
&= \delta'(\hat{\delta}(q_0, w), a) && \text{(ipotesi induttiva)} \\
&= \bigcup_{p \in \hat{\delta}(q_0,w)} \delta(p,a) && \text{(definizione di } \delta') \\
&= \hat{\delta}(q_0, wa) && \text{(definizione di } \hat{\delta} \text{ per gli NFA).}
\end{aligned}$$

[2]Sarebbe più preciso definire $Q' = \{q_1, \ldots, q_{2^{|Q|}}\}$ e stabilire una corrispondenza biunivoca tra questi stati e gli elementi di $\mathcal{P}(Q)$; usare direttamente i sottoinsiemi come stati rende però la dimostrazione molto più snella.

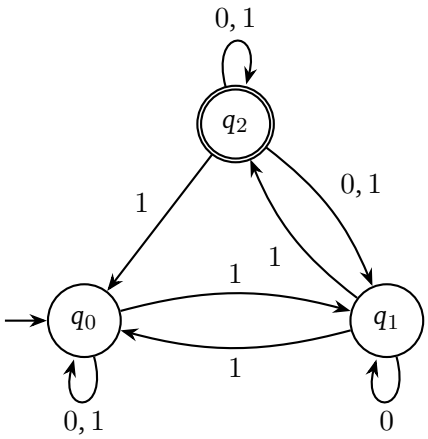
Infine, per ogni $x \in \Sigma^*$,

$$\begin{aligned} x \in \mathcal{L}(\mathcal{M}) &\iff \hat{\delta}'(q_0', x) \in \mathcal{F}' \\ &\iff \hat{\delta}(q_0, x) \in \mathcal{F}' && \text{(uguaglianza appena dimostrata)} \\ &\iff \hat{\delta}(q_0, x) \cap \mathcal{F} \neq \emptyset && \text{(definizione di } \mathcal{F}') \\ &\iff x \in \mathcal{L}(\mathcal{N}), \end{aligned}$$

quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{N})$. □

DFA e NFA riconoscono quindi la stessa classe di linguaggi, i linguaggi regolari: il non determinismo non aumenta la potenza di calcolo. Allo stesso tempo, dare la possibilità di raggiungere più stati diversi leggendo lo stesso simbolo **non** cambia la classe dei linguaggi riconoscibili.

Esempio. Si consideri l'NFA della Figura 3.7, con stato iniziale q_0 e unico stato finale q_2.



δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_1\}$	$\{q_0, q_2\}$
q_2	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$

FIGURA 3.7 *Un NFA e la sua matrice di transizione.*

Si applica la costruzione della dimostrazione (b), che è del tutto meccanica: ogni sottoinsieme di $Q = \{q_0, q_1, q_2\}$ è uno stato del DFA, e la sua riga nella matrice di transizione si ottiene unendo le righe dell'NFA dei suoi elementi. Ad esempio,

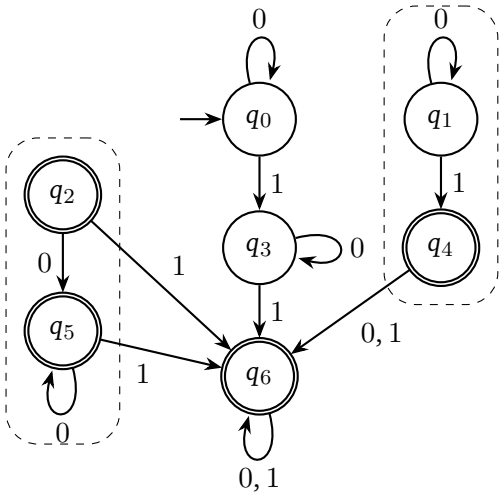
$$\delta'(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0, q_1\} \cup \{q_0, q_2\} = \{q_0, q_1, q_2\}.$$

Gli stati finali sono i sottoinsiemi che contengono q_2, l'unico stato finale dell'NFA. Si ottiene la Tabella 3.1, in cui a ogni sottoinsieme non vuoto si dà un nome q_i'.

STATO	SOTTOINSIEME	0	1	FINALE
	$\emptyset$	$\emptyset$	$\emptyset$	
q_0'	$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$	
q_1'	$\{q_1\}$	$\{q_1\}$	$\{q_0, q_2\}$	
q_2'	$\{q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q_3'	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	
q_4'	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q_5'	$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q_6'	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓

TABELLA 3.1 *Costruzione per sottoinsiemi del DFA equivalente all'NFA della Figura 3.7.*

Sostituendo a ogni sottoinsieme il suo nome, e chiamando per semplicità q_i lo stato q'_i, si ottiene il DFA della Figura 3.8, in cui lo stato $\emptyset$ è omesso.



δ'	0	1
q_0	q_0	q_3
q_1	q_1	q_4
q_2	q_5	q_6
q_3	q_3	q_6
q_4	q_6	q_6
q_5	q_5	q_6
q_6	q_6	q_6

FIGURA 3.8 *Il DFA ottenuto dalla Tabella 3.1; i riquadri tratteggiati racchiudono gli stati non raggiungibili da q_0.*

È interessante notare che alcuni stati non sono raggiungibili da q_0. Gli stati q_1 e q_2 non hanno archi entranti da altri stati, mentre q_4 e q_5 ne hanno, ma solo da q_1 e da q_2. Quello che conta, infatti, non è avere archi entranti, ma l'esistenza di un cammino da q_0: gli unici stati raggiungibili sono q_0, q_3 e q_6.

Gli stati non raggiungibili non cambiano il linguaggio riconosciuto e si possono eliminare, ottenendo il DFA minimo della Figura 3.9.[3] È lo stesso automa dell'esempio sulle stringhe con almeno due 1 (Figura 3.5), con q_3 e q_6 al posto di q_1 e q_2: l'NFA di partenza riconosce quindi proprio quel linguaggio.

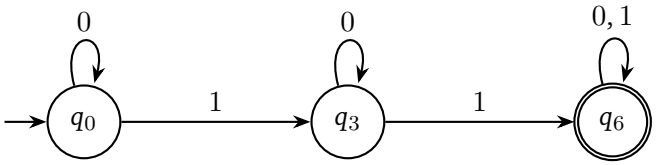


FIGURA 3.9 *Il DFA minimo, ottenuto eliminando gli stati non raggiungibili.*

◇

[3] In questo caso basta eliminare gli stati non raggiungibili; in generale, per ottenere il DFA minimo può servire anche unire stati equivalenti.

3.6 Automi a stati finiti non deterministici con ε-transizioni

Ci si chiede se dare la possibilità di cambiare stato *senza* leggere simboli cambia la classe dei linguaggi riconoscibili.

Definizione (Automa non deterministico con ε-transizioni). *Rispetto agli NFA cambia solo la funzione di transizione δ:* $\langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ *dove:*

- Q *è un insieme finito di stati;*
- Σ *è un alfabeto;*
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ *è la* funzione di transizione*: per ogni input posso raggiungere più stati. Pertanto è una funzione non deterministica. In Input a δ è possibile non avere input per cambiare stato;*
- q_0 *è lo stato* iniziale*;*
- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali

Prima di poter definire $\hat{\delta}$, è necessario definire $\varepsilon\text{-closure}\colon Q \to \mathcal{P}(Q)$. $\varepsilon\text{-closure}(q)$ è l'insieme di tutti gli stati raggiungibili da q senza leggere simboli, ovvero seguendo zero o più archi etichettati con ε:

$$\varepsilon\text{-closure}(q) = \{p \in Q \mid p \text{ è raggiungibile da } q \text{ con zero o più } \varepsilon\text{-transizioni}\}.$$

Equivalentemente, è il più piccolo insieme che contiene q, tutti gli stati p raggingibili da q fornendo come input ε e contiene anche $\delta(p, \varepsilon)$. In particolare $q \in \varepsilon\text{-closure}(q)$, perché q si raggiunge con zero ε-transizioni, e $\delta(q, \varepsilon) \subseteq \varepsilon\text{-closure}(q)$. Si consideri ad esempio $q \xrightarrow{\varepsilon} p \xrightarrow{\varepsilon} r$, anche r appartiene a $\varepsilon\text{-closure}(q)$, pur non appartenendo necessariamente a $\delta(q, \varepsilon)$. Per un insieme di stati $S \subseteq Q$ si pone

$$\varepsilon\text{-closure}(S) = \bigcup_{s \in S} \varepsilon\text{-closure}(s).$$

Quindi adesso è possibile procedere con la definizione della funzione estesa $\hat{\delta}$:

$$\hat{\delta}\colon Q \times \Sigma^* \to \mathcal{P}(Q), \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q), \\ \hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q,x)} \varepsilon\text{-closure}(\delta(p, a)). \end{cases}$$

La ricorsione avviene sulla stringa e non sugli stati: $\hat{\delta}(q, xa)$ richiama $\hat{\delta}(q, x)$ con lo stesso stato di partenza q e la stringa privata dell'ultimo simbolo, fino al caso base $\hat{\delta}(q, \varepsilon)$.

Esempio. Si consideri l'automa della Figura 3.10 e si calcoli $\hat{\delta}(q, a)$. Poiché $a = \varepsilon a$, si applica il secondo caso della definizione con $x = \varepsilon$.

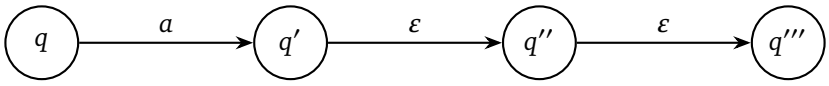


Figura 3.10 *Un automa con due ε-transizioni consecutive.*

Da q non escono archi etichettati con ε, quindi $\hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q) = \{q\}$ e l'unione è su un solo stato. Leggendo a si ottiene $\delta(q, a) = \{q'\}$, e infine si seguono tutte le catene

di archi ε a partire da q':

$$\begin{aligned}\hat{\delta}(q,a) &= \bigcup_{p\in\hat{\delta}(q,\varepsilon)} \varepsilon\text{-closure}(\delta(p,a)) \\ &= \varepsilon\text{-closure}(\{q'\}) \qquad \text{Usando l'estensione sugli Insiemi} \\ &= \{q', q'', q'''\}.\end{aligned}$$

Per calcolare la chiusura si parte da $\{q'\}$ e si aggiungono gli stati raggiungibili con un arco ε da quelli già presenti, finché non se ne trovano di nuovi: prima q'', poi q'''. Da q''' non escono archi ε, quindi ci si ferma. A differenza di DFA e NFA, $\hat{\delta}(q,a)$ può quindi essere diverso da $\delta(q,a) = \{q'\}$.

Le ε-transizioni contano anche prima di leggere un simbolo: in un automa $q \xrightarrow{\varepsilon} r \xrightarrow{a} s$ si ha $\delta(q,a) = \emptyset$, ma $\hat{\delta}(q,\varepsilon) = \{q,r\}$ e quindi $s \in \hat{\delta}(q,a)$. ◇

Esempio. Si calcoli $\hat{\delta}(q,ab)$ per l'automa della Figura 3.11, svolgendo per intero la ricorsione. Le transizioni dell'automa sono $\delta(q,a) = \{q'\}$, $\delta(q',\varepsilon) = \{q''\}$, $\delta(q'',b) = \{s\}$ e $\delta(s,\varepsilon) = \{t\}$; tutte le altre danno $\emptyset$.

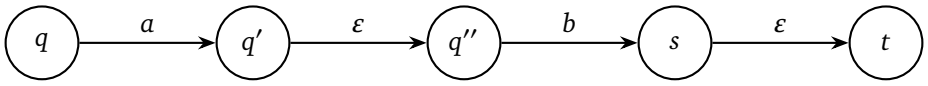


Figura 3.11 *Un automa in cui il simbolo b si può leggere solo dopo una ε-transizione.*

Discesa. Ogni chiamata toglie l'ultimo simbolo della stringa, fino al caso base:

$$\begin{aligned}\hat{\delta}(q,ab) &\qquad ab = a\cdot b\text{: si richiama } \hat{\delta}(q,a),\\ \hat{\delta}(q,a) &\qquad a = \varepsilon\cdot a\text{: si richiama } \hat{\delta}(q,\varepsilon),\\ \hat{\delta}(q,\varepsilon) &\qquad \text{caso base.}\end{aligned}$$

Risalita. Ogni livello legge il proprio simbolo e chiude con le ε-transizioni. Il caso base restituisce

$$\hat{\delta}(q,\varepsilon) = \varepsilon\text{-closure}(q) = \{q\},$$

Il livello intermedio legge a:

$$\begin{aligned}\hat{\delta}(q,a) &= \bigcup_{p\in\{q\}} \varepsilon\text{-closure}(\delta(p,a)) && \text{(unione su un solo stato)}\\ &= \varepsilon\text{-closure}(\delta(q,a)) = \varepsilon\text{-closure}(\{q'\}) \\ &= \{q', q''\} && (\delta(q',\varepsilon) = \{q''\},\ \delta(q'',\varepsilon) = \emptyset).\end{aligned}$$

Il livello più esterno legge b, partendo dall'insieme appena restituito:

$$\begin{aligned}\hat{\delta}(q,ab) &= \bigcup_{p\in\{q',q''\}} \varepsilon\text{-closure}(\delta(p,b)) \\ &= \varepsilon\text{-closure}(\delta(q',b)) \cup \varepsilon\text{-closure}(\delta(q'',b)) && \text{(unione su due stati)}\\ &= \varepsilon\text{-closure}(\emptyset) \cup \varepsilon\text{-closure}(\{s\}) \\ &= \emptyset \cup \{s,t\} = \{s,t\} && (\delta(s,\varepsilon) = \{t\},\ \delta(t,\varepsilon) = \emptyset).\end{aligned}$$

A ogni livello l'insieme su cui si fa l'unione è il valore restituito dal livello sottostante: $\{q\}$ per la lettura di a, $\{q', q''\}$ per la lettura di b. Il ramo $p = q'$ non contribuisce, perché da q' non si legge b; è il ramo $p = q''$, raggiunto solo grazie alla ε-closure applicata dopo a, a portare in s. ◇

Teorema 4 (Equivalenza tra NFA e ε-NFA).

a. *Sia $\mathcal{N}_\varepsilon = \langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ un ε-NFA. Allora esiste un NFA $\mathcal{N} = \langle Q', \Sigma', \delta', q_0', \mathcal{F}'\rangle$ tale per cui $\mathcal{L}(\mathcal{N}_\varepsilon) = \mathcal{L}(\mathcal{N})$.*
b. *Sia $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ un ε-NFA. Allora esiste un NFA $\mathcal{N}_\varepsilon = \langle Q', \Sigma', \delta', q_0', \mathcal{F}'\rangle$ tale per cui $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{N}_\varepsilon)$.*

Dimostrazione del Teorema 4. Anche in questo caso di deve mostrare un'equivalenza. Pertanto, la dimostrazione richiede entrambi i versi. L'esistenza di un ε-NFA dato un NFA è banale, dato che un NFA è un ε-NFA dove non vi sono archi etichettati con ε.

Da ε-NFA a NFA (Costruzione formale)

La dimostrazione richiede teoricamente tre passaggi:

1. Costruire un NFA equivalente definito come $\mathcal{N} = \langle Q', \Sigma, \delta', q_0, F'\rangle$.
2. Dimostrare che l'intersezione tra la chiusura transitiva dell'automa originale e i suoi stati finali equivale a quella del nuovo automa: $\hat{\delta}(q_0, x) \cap F = \hat{\delta}'(q_0, x) \cap F'$.
3. Dimostrare l'uguaglianza $\hat{\delta}(q_0, x) = \hat{\delta}'(q_0', x)$ sfruttando il punto precedente.

Regole di trasformazione

- L'insieme degli stati ($Q' = Q$), l'alfabeto ($\Sigma' = \Sigma$) e lo stato iniziale ($q_0' = q_0$) restano invariati rispetto all'automa originale.
- L'insieme degli stati finali F' ingloba lo stato iniziale se, partendo da esso, è possibile raggiungere un nodo di accettazione originale tramite soli passi ε:

$$F' = \begin{cases} F \cup \{q_0\} & \text{se } \varepsilon\text{-closure}(q_0) \cap F \neq \emptyset \\ F & \text{altrimenti} \end{cases}$$

- La nuova funzione di transizione ingloba le ε-transizioni originali direttamente nelle transizioni di base:

$$\delta'(q, a) = \hat{\delta}(q, a) \stackrel{\text{def}}{=} \varepsilon\text{-closure}(\delta(q, a))$$

Sarebba anche da dimostrare che [4]

$$\hat{\delta}'(q_0', x) \cap \mathcal{F}' \neq \emptyset \iff \hat{\delta}(q_0, x) \cap \mathcal{F} \neq \emptyset$$

□

3.7 Operazioni Tra Linguaggi

Siano $\mathcal{L}, \mathcal{L}_1, \mathcal{L}_2$ linguaggi sull'alfabeto Σ. Di seguito tutte le operazioni applicabili su linguaggi:

- Concatenazione:

$$\mathcal{L}_1 \cdot \mathcal{L}_2 = \{\, x \cdot y \mid x \in \mathcal{L}_1 \wedge y \in \mathcal{L}_2 \,\}$$

Si noti che:

$$\varepsilon \in \mathcal{L}_i \implies \mathcal{L}_i \subseteq \mathcal{L}_1 \cdot \mathcal{L}_2 \qquad i \in \{1, 2\}$$

[4] a lezione non è stato visto

- Unione: $\mathcal{L}_1, \mathcal{L}_2$ su Σ:

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{ x \mid x \in \mathcal{L}_1 \vee x \in \mathcal{L}_2 \}$$

- Chiusura di Kleene: $\mathcal{L}$ linguaggio su Σ:

$$\text{Concatenazione Iterativa} : \begin{cases} \mathcal{L}^0 = \{\varepsilon\} \\ \mathcal{L}^{i+1} = \mathcal{L} \cdot \mathcal{L}^i \end{cases}$$

$$\mathcal{L}^* = \bigcup_{i \geq 0} \mathcal{L}^i \qquad \mathcal{L}^+ = \bigcup_{i \geq 1} \mathcal{L}^i$$

Esempio. sia $\mathcal{L} = \{ab\}$. Allora:

$$\begin{aligned} \mathcal{L}^0 &= \{\varepsilon\} \\ \mathcal{L}^1 &= \{ab\} \cdot \{\varepsilon\} = \{ab\} \\ \mathcal{L}^2 &= \{ab\} \cdot \mathcal{L}^1 = \{ab\}\{ab\} = \{abab\} \\ &\vdots \end{aligned}$$

Invece, con $\mathcal{L} = \{ab, cd\}$:

$$\begin{aligned} \mathcal{L}^0 &= \{\varepsilon\} \\ \mathcal{L}^1 &= \{ab, cd\} \cdot \{\varepsilon\} = \{ab, cd\} \\ \mathcal{L}^2 &= \{ab, cd\} \cdot \mathcal{L}^1 = \{abab, abcd, cdab, cdcd\} \\ &\vdots \end{aligned}$$

◇

3.8 Espressioni Regolari

Definizione (Espressioni Regolari). *Un'**espressione regolare** (RE) è uno strumento formale utilizzato per descrivere e generare in modo compatto le stringhe che appartengono a un linguaggio regolare.*

Le RE vengono definite in modo induttivo:

- *$\emptyset$ è un'espressione regolare che denota l'insieme vuoto.*
- *ε è un'espressione regolare che denota l'insieme $\{\varepsilon\}$.*
- *Per ogni simbolo $a \in \Sigma$, a è un'espressione regolare che denota l'insieme $\{a\}$.*
- *Se r e s sono espressioni regolari denotanti rispettivamente gli insiemi R ed S, allora $(r + s)$, $(r \cdot s)$, e (r^*) sono espressioni regolari che denotano gli insiemi $R \cup S$, $R \cdot S$, e R^* rispettivamente.*

Se r è una espressione regolare, indicheremo con $\mathcal{L}(r)$ il linguaggio denotato da r. Tra espressioni regolari valgono anche uguaglianza che permettono la loro manipolazione algebrica. In particolare:

$$r = s \iff \mathcal{L}(r) = \mathcal{L}(s)$$

Esempio. Sia $\Sigma = \{0, 1\}$. Il linguaggio associato all'espressione regolare $r = 1(0 + 1)^*00$. Allora:

$$\mathcal{L}(r) = \mathcal{L}(1)(\mathcal{L}(0) \cup \mathcal{L}(1))^* \mathcal{L}(0)\mathcal{L}(0) = \{1x00 : x \in \Sigma^*\}$$

Ossia tutte le stringhe che sono (in binario) un multiplo di 4. ◇

Teorema 5 (Equivalenza tra DFA e ER (McNaughton e Yamada, 1960)). *Sia r un espressione regolare, allora esiste un ε-DFA $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{M})$.*

Dimostrazione del Teorema 5. Si procede per induzione, costruendo per ogni caso base e per le applicazioni dei passi induttivi. Caso Base

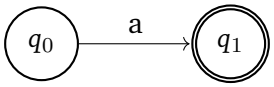
- se $r = \varepsilon$, allora l'automa seguente riconosce $\mathcal{L}(r) = \{\varepsilon\}$.



- se $r = \emptyset$, allora l'automa seguente riconosce $\mathcal{L}(r) = \emptyset$.

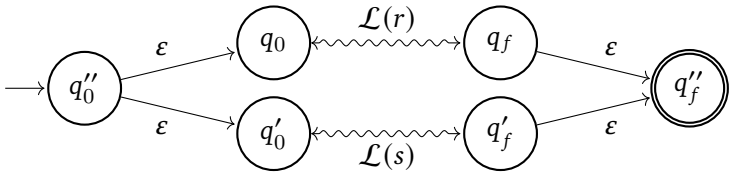


- se $r = a$, allora l'automa seguente riconosce $\mathcal{L}(r) = \emptyset$.

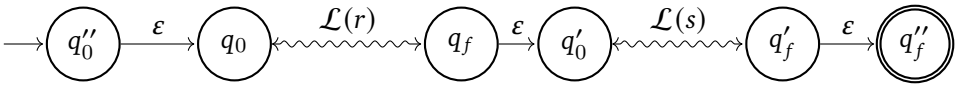


Passo Induttivo Anche qui ci sono diversi casi da analizzare:

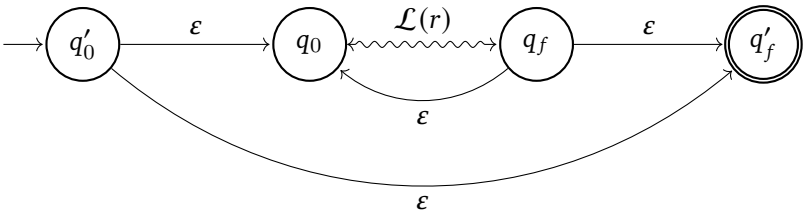
- Sia $r = r_1 + r_2$. Per $i \in \{1, 2\}$, sia $\mathcal{M}_i$, con stato iniziale q_0^i e stato finale q_f^i che riconoscono il linguaggio $\mathcal{L}(r_i)$. L'esistenza di tali automi è garantita per ipotesi induttiva. Il seguente automa, con stato iniziale q_0 e stato finale q_f riconosce il linguaggio $\mathcal{L}(r) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$



- Sia $r = r_1 r_2$. Per $i \in \{1, 2\}$, sia $\mathcal{M}_i$, con stato iniziale q_0^i e stato finale q_f^i l'automa che riconosce $\mathcal{L}(r_i)$. L'esistenza di tali automi è assicurata dall'ipotesi induttiva. Il seguente automa con stato iniziale q_0 e stato finale q_f riconosce il linguaggio $\mathcal{L}(r_1)\mathcal{L}(r_2)$:



- $\mathcal{L}(r) = (\mathcal{L}(r_1))^*$:
  Sia $\mathcal{M}_1$, con stato iniziale q_0^1 e stato finale q_f^1 l'automa che riconosce $\mathcal{L}(r_1)$.
  L'esistenza di tale automa è assicurata dall'ipotesi induttiva. Il seguente automa, con stato iniziale q_0 e stato finale q_f riconosce $\mathcal{L}(r) = (\mathcal{L}(r_1))^*$:

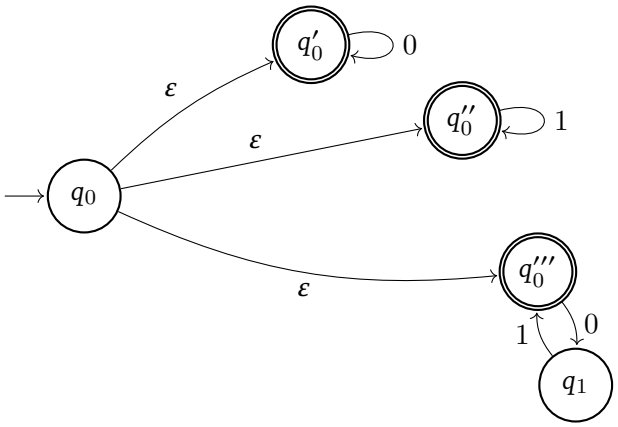


□

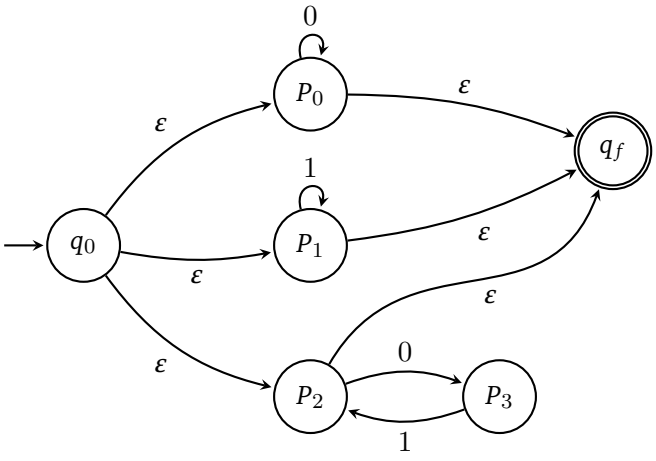
Esempio. Si consideri l'espressione regolare $r = 0^m + 1^m + (01)^m$:

$$\mathcal{L}(r) = \{x \in \{0,1\}^* \mid \exists m \in \mathbb{N} \,.\, x = 0^m \vee x = 1^m \vee x = (01)^m\}$$

Il seguente automa (ad-hoc) riconosce correttamenre il linguaggio, ma non segue la costruzione vista nella dimostrazione del teorema.



Questo invece è costruito seguendo formalmente i passi induttivi della dimostrazione del teorema.



◇

Ricapitolando:

$$\text{Linguaggi Regolari} \iff \text{DFA} \iff \text{NFA} \iff \varepsilon\text{-NFA} \iff \text{ER}$$

3.9 Proprietà dei Linguaggi Regolari

I linguaggi regolari godono di alcune proprietà di chiusura. Sia LR l'insieme dei linguaggi regolari:

$$\text{LR} = \{\mathcal{L} \subseteq \Sigma^* \mid \text{“}\mathcal{L} \text{ è un linguaggio regolare”}\}.$$

LR è chiuso rispetto all'*unione*, alla *concatenazione* e alla *stella di Kleene*:[5]

$$\mathcal{L}_1, \mathcal{L}_2 \in \text{LR} \implies \begin{cases} \mathcal{L}_1 \cup \mathcal{L}_2 \in \text{LR} \\ \mathcal{L}_1 \cdot \mathcal{L}_2 \in \text{LR} \\ \mathcal{L}_1^* \in \text{LR}. \end{cases}$$

Inoltre, LR è chiuso anche rispetto alla *complementazione*:[6]

$$\mathcal{L} \in \text{LR} \implies \overline{\mathcal{L}} = \mathcal{L} \setminus \Sigma^* \in \text{LR}.$$

Per le leggi di De Morgan, la chiusura rispetto all'unione e alla complementazione implica anche la chiusura rispetto all'*intersezione*:

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \overline{\overline{\mathcal{L}_1} \cup \overline{\mathcal{L}_2}} \implies \mathcal{L}_1 \cap \mathcal{L}_2 \in \text{LR}.$$

Nota. La proprietà di chiusura rispetto all'unione e all'intersezione si applica solo quando il numero di insiemi che si stanno considerando è finito. Questo significa che, anche se i singoli L_n sono regolari:

$$\forall m \in \mathbb{N}, \mathcal{L}_m \text{ è regolare} \not\implies \bigcup_{m \in \mathbb{N}} \mathcal{L}_m \text{ è regolare}$$
$$\forall m \in \mathbb{N}, \mathcal{L}_m \text{ è regolare} \not\implies \bigcap_{m \in \mathbb{N}} \mathcal{L}_m \text{ è regolare}$$

[5] La chiusura rispetto a queste operazioni deriva direttamente dall'equivalenza con le espressioni regolari.

[6] L'operazione di complementazione si traduce in un DFA nella semplice ridefinizione dell'insieme degli stati finali come $F' = Q \setminus F$.

3.10 Problemi di Decidibilità

Sia $\mathcal{L}$ un linguaggio regolare. I seguenti problemi sono **decidibili**:[7]

1. **Vacuità** ($\mathcal{L} \neq \emptyset$):

$$\mathcal{L} \neq \emptyset \iff \exists x \in \mathcal{L}(\mathcal{M}) \quad \text{t.c.} \quad |x| < |Q|$$

   dove $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ è l'automa tale che $\mathcal{L} = \mathcal{L}(\mathcal{M})$.

2. **Infinità** ($|\mathcal{L}| = \omega$):

$$|\mathcal{L}| = \omega \iff \exists x \in \mathcal{L}(\mathcal{M}) \quad \text{t.c.} \quad n \leq |x| < 2n$$

   dove $n = |Q|$ è il numero di stati dell'automa $\mathcal{M}$.

3. **Equivalenza**:

   Dati $\mathcal{M}_1$ e $\mathcal{M}_2$, l'uguaglianza $\mathcal{L}(\mathcal{M}_1) = \mathcal{L}(\mathcal{M}_2)$ è decidibile.

3.11 Relazioni di Equivalenza e Linguaggi

Relazione di equivalenza indotta da $\mathcal{L}$

Sia $\mathcal{L} \subseteq \Sigma^*$. Definiamo $R_{\mathcal{L}} \subseteq \Sigma^* \times \Sigma^*$:

$$\forall x, y \in \Sigma^*, \quad x \, R_{\mathcal{L}} \, y \iff (\forall z \in \Sigma^*, \; xz \in \mathcal{L} \iff yz \in \mathcal{L})$$

Relazione di equivalenza indotta da $\mathcal{M}$ (DFA)

Sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$. Definiamo $R_{\mathcal{M}} \subseteq \Sigma^* \times \Sigma^*$:

$$\forall x, y \in \Sigma^*, \quad x \, R_{\mathcal{M}} \, y \iff \hat{\delta}(q_0, x) = \hat{\delta}(q_0, y)$$

Relazione invariante a destra

Sia $R \subseteq \Sigma^* \times \Sigma^*$ una relazione di equivalenza. R è **invariante a destra** se e solo se:

$$x \, R \, y \implies \forall z \in \Sigma^*, \; xz \, R \, yz$$

In particolare $R_{\mathcal{L}}$ è invariante a destra.

Linguaggio indotto da uno stato

$$\mathcal{L}_q = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) = q\}$$

è il linguaggio "accettato" dallo stato q.

[7] Un problema è decidibile se esiste un algoritmo che, ricevuta in input un'istanza qualsiasi del problema, termina sempre in un numero finito di passi e fornisce la risposta corretta (sì/no).

4 Esercizi

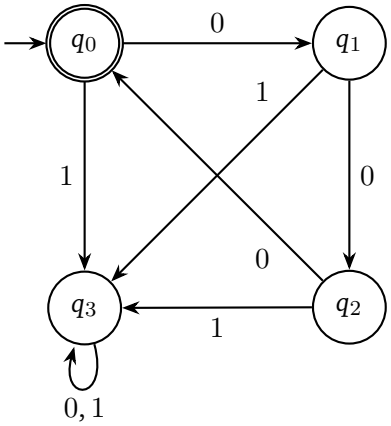
4.1 Automi a stati finiti deterministici

Gli esercizi seguenti dimostrano che un linguaggio è regolare costruendo un DFA che lo riconosce, con il metodo della Sezione 3.4.1.

Esercizio 1

Si dimostri che il linguaggio $\mathcal{L} = \{0^{3n} \mid n \geq 0\}$ *sull'alfabeto* $\Sigma = \{0, 1\}$ *è regolare.*

Svolgimento. Il linguaggio contiene le stringhe formate solo da zeri, in numero multiplo di 3. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\} \rangle$ della Figura 4.1: gli stati q_0, q_1 e q_2 contano gli zeri letti modulo 3, mentre q_3 è lo stato in cui si finisce, senza più uscirne, appena si legge un 1.



δ	0	1
q_0	q_1	q_3
q_1	q_2	q_3
q_2	q_0	q_3
q_3	q_3	q_3

Figura 4.1 *DFA che riconosce* $\{0^{3n} \mid n \geq 0\}$ *e la sua matrice di transizione.*

Bisogna dimostrare che $\mathcal{L}(\mathcal{M}) = \mathcal{L}$, cioè

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F}, \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F}. \end{cases}$$

Come nell'esempio della Sezione 3.4.1, la seconda condizione non basta per il passo induttivo, perché non dice in quale dei tre stati non finali si trovi l'automa. La si scorpora

quindi secondo il significato degli stati, ottenendo la proprietà più forte

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_0, & (1) \\ x \notin \mathcal{L} \wedge x = 0^{3i+1} \implies \hat{\delta}(q_0, x) = q_1, & (2) \\ x \notin \mathcal{L} \wedge x = 0^{3i+2} \implies \hat{\delta}(q_0, x) = q_2, & (3) \\ x \notin \mathcal{L} \wedge \text{“}x \text{ contiene almeno un } 1\text{”} \implies \hat{\delta}(q_0, x) = q_3, & (4) \end{cases}$$

con $i \geq 0$. Le quattro condizioni coprono tutte le stringhe e si escludono a vicenda: una stringa o contiene almeno un 1, oppure è della forma 0^k, e in tal caso k diviso per 3 dà resto 0, 1 o 2.

Per $|x| \leq 2$ esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$, quindi si sceglie questa come base:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \in \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 & \text{(caso 1 di } \Psi) \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 & \text{(caso 2 di } \Psi) \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_3 & \text{(caso 4 di } \Psi) \\ |x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 3 di } \Psi) \\ & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_3 & \text{(caso 4 di } \Psi) \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_3 & \text{(caso 4 di } \Psi) \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_3 & \text{(caso 4 di } \Psi) \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle quattro condizioni. Si distingue un caso per ciascuna di esse.

1. $x' \in \mathcal{L}$, cioè $x' = 0^{3i}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   1a. $x = x'0 = 0^{3i+1}$, quindi $x \notin \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$
   1b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_0, 1) = q_3.$$
2. $x' \notin \mathcal{L}$ e $x' = 0^{3i+1}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   2a. $x = x'0 = 0^{3i+2}$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   2b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_1, 1) = q_3.$$
3. $x' \notin \mathcal{L}$ e $x' = 0^{3i+2}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$.
   3a. $x = x'0 = 0^{3i+3} = 0^{3(i+1)}$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_2, 0) = q_0.$$

3b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

$$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_2, 1) = q_3.$$

4. $x' \notin \mathcal{L}$ e x' contiene almeno un 1: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_3$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, anche $x = x'a$ contiene almeno un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

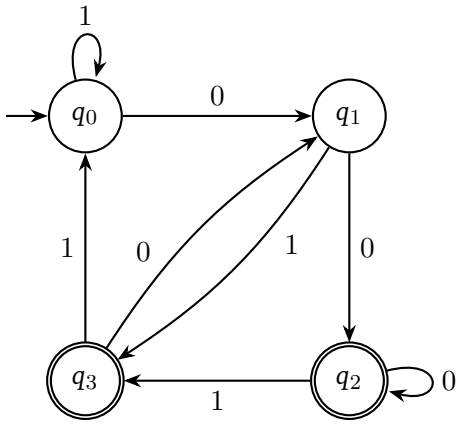
$$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_3, a) = q_3.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale anche per $|x| = n + 1$ e, per induzione, per ogni $x \in \Sigma^*$. In particolare, se $x \in \mathcal{L}$ allora $\hat{\delta}(q_0, x) = q_0 \in \mathcal{F}$; se $x \notin \mathcal{L}$ allora $\hat{\delta}(q_0, x) \in \{q_1, q_2, q_3\}$, e nessuno di questi stati è finale. Quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 2

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid \exists v \in \Sigma^*,\ a \in \Sigma.\ x = v0a\}$ sull'alfabeto $\Sigma = \{0, 1\}$, formato dalle stringhe il cui penultimo simbolo è 0, è regolare.

SVOLGIMENTO. Lo riconosce il DFA $\mathcal{M} = \langle\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_2, q_3\}\rangle$ della Figura 4.2. Ogni stato ricorda gli ultimi due simboli letti: q_0 corrisponde a 11, q_1 a 10, q_2 a 00 e q_3 a 01. Gli stati finali sono quelli in cui il penultimo simbolo è 0, cioè q_2 e q_3.



δ	0	1
q_0	q_1	q_0
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_1	q_0

FIGURA 4.2 *DFA che riconosce le stringhe con penultimo simbolo 0 e la sua matrice di transizione.*

Le stringhe con meno di due simboli non hanno un penultimo simbolo, quindi non appartengono a $\mathcal{L}$. Per le stringhe con almeno due simboli si scorpora la tesi secondo gli ultimi due simboli letti, ottenendo

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \wedge x = v00 \implies \hat{\delta}(q_0, x) = q_2, & (1) \\ x \in \mathcal{L} \wedge x = v01 \implies \hat{\delta}(q_0, x) = q_3, & (2) \\ x \notin \mathcal{L} \wedge x = v10 \implies \hat{\delta}(q_0, x) = q_1, & (3) \\ x \notin \mathcal{L} \wedge x = v11 \implies \hat{\delta}(q_0, x) = q_0, & (4) \end{cases}$$

con $v \in \Sigma^*$. Per $|x| \geq 2$ le quattro condizioni coprono tutte le stringhe e si escludono a vicenda, perché dipendono solo dagli ultimi due simboli.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$. Quelle più corte non soddisfano nessuna condizione di Ψ e si verificano direttamente sulla tesi:

$$\left.\begin{array}{llll} |x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} & \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 \notin \mathcal{F} & \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_0 \notin \mathcal{F} & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 1 di } \Psi) \\ & x = 01 \in \mathcal{L} & \hat{\delta}(q_0, 01) = q_3 & \text{(caso 2 di } \Psi) \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 3 di } \Psi) \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_0 & \text{(caso 4 di } \Psi) \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $2 \leq |x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle quattro condizioni. Gli ultimi due simboli di x sono l'ultimo simbolo di x' seguito da quello appena letto.

1. $x' \in \mathcal{L}$ e $x' = v00$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$.
   1a. $x = x'0 = (v0)00$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_2, 0) = q_2.$$
   1b. $x = x'1 = (v0)01$, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_2, 1) = q_3.$$
2. $x' \in \mathcal{L}$ e $x' = v01$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_3$.
   2a. $x = x'0 = (v0)10$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_3, 0) = q_1.$$
   2b. $x = x'1 = (v0)11$, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_3, 1) = q_0.$$
3. $x' \notin \mathcal{L}$ e $x' = v10$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   3a. $x = x'0 = (v1)00$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   3b. $x = x'1 = (v1)01$, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_1, 1) = q_3.$$
4. $x' \notin \mathcal{L}$ e $x' = v11$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   4a. $x = x'0 = (v1)10$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$

4b. $x = x'1 = (v1)11$, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

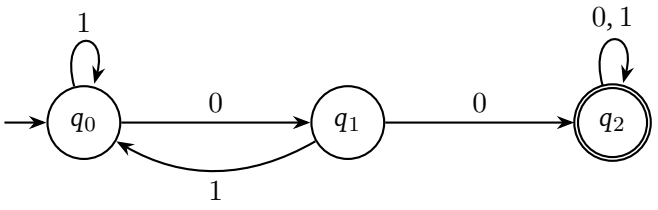
$$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_0, 1) = q_0.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale per ogni x con $|x| \geq 2$. Ne segue la tesi: se $x \in \mathcal{L}$, allora $|x| \geq 2$ e x termina con 00 o 01, quindi $\hat{\delta}(q_0, x) \in \{q_2, q_3\} = \mathcal{F}$; se $x \notin \mathcal{L}$, allora x ha meno di due simboli, e la base mostra che $\hat{\delta}(q_0, x) \notin \mathcal{F}$, oppure termina con 10 o 11, e $\hat{\delta}(q_0, x) \in \{q_0, q_1\}$, stati non finali. Quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 3

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid$ "x contiene almeno due 0 consecutivi"$\}$ sull'alfabeto $\Sigma = \{0, 1\}$ è regolare.

SVOLGIMENTO. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\} \rangle$ della Figura 4.3. Finché non ha letto due 0 consecutivi l'automa ricorda soltanto l'ultimo simbolo: è in q_1 se l'ultimo simbolo è 0 e in q_0 altrimenti. Appena legge il secondo 0 consecutivo passa in q_2, da cui non esce più.



δ	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

FIGURA 4.3 *DFA che riconosce le stringhe con almeno due 0 consecutivi e la sua matrice di transizione.*

Si scorpora la tesi secondo il significato degli stati, ottenendo

$$\Psi(x) = \begin{cases} x \notin \mathcal{L} \wedge x = w0 \implies \hat{\delta}(q_0, x) = q_1, & (1) \\ x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, & (2) \\ x \notin \mathcal{L} \wedge x = w1 \implies \hat{\delta}(q_0, x) = q_0, & (3) \end{cases}$$

con $w \in \Sigma^*$. Le tre condizioni si escludono a vicenda e coprono tutte le stringhe tranne ε, che non ha un ultimo simbolo e si verifica a parte.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} & & \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 & \text{(caso 1 di } \Psi) & \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_0 & \text{(caso 3 di } \Psi) & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 2 di } \Psi) & \\ & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_0 & \text{(caso 3 di } \Psi) & \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 1 di } \Psi) & \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_0 & \text{(caso 3 di } \Psi) & \end{array}\right\} \text{BASE}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle tre condizioni.

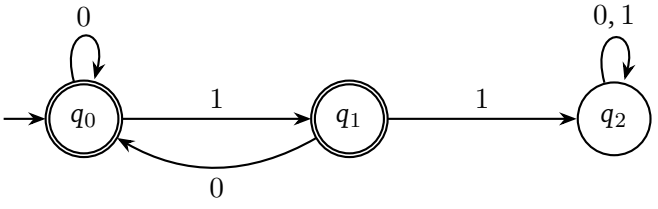
1. $x' \notin \mathcal{L}$ e $x' = w0$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   1a. $x = x'0 = w00$ contiene due 0 consecutivi, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   1b. $x = x'1 = w01$: aggiungere un 1 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 1 (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_1, 1) = q_0.$$
2. $x' \in \mathcal{L}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, $x = x'a$ contiene ancora i due 0 consecutivi di x', quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_2, a) = q_2.$$
3. $x' \notin \mathcal{L}$ e $x' = w1$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   3a. $x = x'0 = w10$: poiché x' termina con 1, aggiungere uno 0 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 0 (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$
   3b. $x = x'1 = w11$: aggiungere un 1 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 1 (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_0.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$. Quindi $\hat{\delta}(q_0, x) = q_2 \in \mathcal{F}$ se $x \in \mathcal{L}$, mentre se $x \notin \mathcal{L}$ lo stato raggiunto è q_0 o q_1, che non sono finali: $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 4

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid$ "x non contiene due 1 consecutivi"$\}$ sull'alfabeto $\Sigma = \{0, 1\}$ è regolare.

Svolgimento. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_0, q_1\} \rangle$ della Figura 4.4. Finché non ha letto due 1 consecutivi l'automa ricorda soltanto l'ultimo simbolo: è in q_1 se l'ultimo simbolo è 1 e in q_0 altrimenti. Appena legge il secondo 1 consecutivo passa in q_2, da cui non esce più e che non è finale.



δ	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_2

Figura 4.4 *DFA che riconosce le stringhe senza due 1 consecutivi e la sua matrice di transizione.*

Si scorpora la tesi secondo il significato degli stati, ottenendo

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \wedge x = w0 \implies \hat{\delta}(q_0, x) = q_0, & (1) \\ x \in \mathcal{L} \wedge x = w1 \implies \hat{\delta}(q_0, x) = q_1, & (2) \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, & (3) \end{cases}$$

con $w \in \Sigma^*$. Anche qui le tre condizioni si escludono a vicenda e coprono tutte le stringhe tranne ε, che si verifica a parte.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \in \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \in \mathcal{F} & & \\ |x| = 1 & x = 0 \in \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 1 \in \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 & \text{(caso 2 di } \Psi) & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 01 \in \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 & \text{(caso 2 di } \Psi) & \\ & x = 10 \in \mathcal{L} & \hat{\delta}(q_0, 10) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 & \text{(caso 3 di } \Psi) & \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle tre condizioni.

1. $x' \in \mathcal{L}$ e $x' = w0$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   1a. $x = x'0 = w00$: aggiungere uno 0 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_0, 0) = q_0.$$
   1b. $x = x'1 = w01$: poiché x' termina con 0, aggiungere un 1 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_1.$$
2. $x' \in \mathcal{L}$ e $x' = w1$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   2a. $x = x'0 = w10$: aggiungere uno 0 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_0.$$
   2b. $x = x'1 = w11$ termina con due 1 consecutivi, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_1, 1) = q_2.$$
3. $x' \notin \mathcal{L}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, $x = x'a$ contiene ancora i due 1 consecutivi di x', quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_2, a) = q_2.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$. Quindi se $x \in \mathcal{L}$ lo stato raggiunto è q_0 o q_1, entrambi finali, mentre se $x \notin \mathcal{L}$ è $q_2 \notin \mathcal{F}$: $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.